

ReBoot: Encrypted Training of Deep Neural Networks with CKKS Bootstrapping

Alberto Pirillo
Politecnico di Milano
Milan, Italy

alberto.pirillo@mail.polimi.it

Luca Colombo
Politecnico di Milano
Milan, Italy

luca2.colombo@polimi.it

Abstract

Growing concerns over data privacy underscore the need for deep learning methods capable of processing sensitive information without compromising confidentiality. Among privacy-enhancing technologies, Homomorphic Encryption (HE) stands out by providing post-quantum cryptographic security and end-to-end data protection, safeguarding data even during computation. While Deep Neural Networks (DNNs) have gained attention in HE settings, their use has largely been restricted to encrypted inference. Prior research on encrypted training has primarily focused on logistic regression or has relied on multi-party computation to enable model fine-tuning. This stems from the substantial computational overhead and algorithmic complexity involved in DNNs training under HE. In this paper, we present ReBoot, the first framework to enable fully encrypted and non-interactive training of DNNs. Built upon the CKKS scheme, ReBoot introduces a novel HE-compliant neural network architecture based on local error signals, specifically designed to minimize multiplicative depth and reduce noise accumulation. ReBoot employs a tailored packing strategy that leverages real-number arithmetic via *SIMD* operations, significantly lowering both computational and memory overhead. Furthermore, by integrating approximate bootstrapping, ReBoot learning algorithm supports effective training of arbitrarily deep multi-layer perceptrons, making it well-suited for machine learning as-a-service. ReBoot is evaluated on both image recognition and tabular benchmarks, achieving accuracy comparable to 32-bit floating-point plaintext training while enabling fully encrypted training. It improves test accuracy by up to +3.27% over encrypted logistic regression, and up to +6.83% over existing encrypted DNN frameworks, while reducing training latency by up to 8.83×. ReBoot is made available to the scientific community as a public repository.

Keywords

Homomorphic Encryption, CKKS, Encrypted Training, Bootstrapping, Multi-Layer Perceptrons

1 Introduction

The increase concerns over data protection have sparked widespread interest in the integration of privacy enhancing technologies into Machine Learning (ML) and Deep Learning (DL) applications. Among these technologies, Homomorphic Encryption (HE) stands out for its unique ability to support computations directly on encrypted data [1]. By offering post-quantum cryptographic security, HE enables data processing in untrusted environments, such as cloud services, without ever exposing sensitive information,

thereby preserving confidentiality throughout computation [14]. However, despite its potential, HE introduces significant challenges that must be addressed. For example, it typically supports only a limited subset of arithmetic operations on encrypted data, primarily additions and multiplications, which restricts computations to polynomial functions. As a result, common ML and DL operations, such as ReLU and sigmoid, are not supported in the encrypted domain. This limitation requires the use of polynomial approximations and the design of alternative model architectures specifically tailored for encrypted computation. Moreover, HE supports only a limited number of consecutive encrypted operations, as each operation introduces additional noise into the ciphertext. To preserve the integrity of encrypted data, a noise threshold is defined, beyond which the ciphertext can no longer be correctly decrypted.

To address this constraint, Fully Homomorphic Encryption (FHE) schemes employ bootstrapping [30], a homomorphic operation that refreshes the ciphertext structure and reduces accumulated noise, thereby enabling unbounded computation. More recently, an *approximate* variant of bootstrapping has been adapted for Leveled Homomorphic Encryption (LHE) schemes such as Cheon-Kim-Kim-Song (CKKS) [19, 26]. Unlike in FHE schemes such as Brakerski-Fan-Vercauteren (BFV) [27] and Torus FHE (TFHE) [20], approximate bootstrapping does not enable unlimited computation, as it does not reduce accumulated noise [4]. Nevertheless, it significantly extends the number of encrypted operations that can be performed by restoring the ciphertext structure.

Bootstrapping plays a crucial role in enabling end-to-end encrypted learning. However, training ML models, and particularly DL models, remains a major open research challenge. This is mainly due to three factors: (i) the inherent noise accumulation of HE, (ii) the significant computational overhead of homomorphic operations, and (iii) the functional limitations imposed by HE schemes. Consequently, existing research has largely focused on encrypted training of simpler ML models [9, 15, 24, 32, 36], or on fine-tuning the final layer of Deep Neural Networks (DNNs) within transfer learning frameworks [3, 35, 37, 47, 57, 61]. More recent work has explored training entire DNNs over encrypted data by leveraging FHE schemes [23, 39, 42, 44, 60]. However, some of these approaches rely on integer-based encryption, which necessitates complex learning algorithms that often compromise model accuracy [49], while others are constrained by multi-party computation mechanisms [41]. To date, no prior work has addressed the challenge of training DNNs on encrypted data using the CKKS scheme in a fully non-interactive setting.

In this regard, the aim of this paper is to address the following research question: *how can CKKS approximate bootstrapping be effectively leveraged to enable fully encrypted interaction-free training of*

DNNs? To this end, we propose ReBoot, a novel encrypted learning framework specifically designed to train HE-compliant Multi-Layer Perceptrons (MLPs) [10] on encrypted data. ReBoot components are specifically designed to manage noise accumulation while minimizing both computational and memory overhead associated with HE. To achieve this, ReBoot introduces a novel HE-compliant NN architecture along with an advanced packing technique that exploits CKKS ability to perform *Single Instruction, Multiple Data (SIMD)* operations within ciphertexts. By integrating approximate bootstrapping, ReBoot learning algorithm supports deep encrypted computation pipelines, allowing fully non-interactive training of MLPs and making it well-suited for practical and scalable machine learning as-a-service scenarios. To the best of our knowledge, ReBoot is the first framework in the literature to extend CKKS-based training beyond logistic regression and fine-tuning, enabling end-to-end encrypted training of real-valued DNNs. ReBoot is implemented as an open-source Python library¹ based on OpenFHE [7].

In summary, our contributions are:

- (1) The design of an advanced packing technique that minimizes both computational and memory overhead of HE by fully exploiting CKKS *SIMD* capabilities.
- (2) The design of a CKKS-compliant MLP architecture in which all operations are optimized to manage noise accumulation.
- (3) The design of ReBoot encrypted learning algorithm, which enables non-interactive encrypted training of HE-compliant MLPs on encrypted data.

Experimental results on state-of-the-art image recognition and tabular benchmarks demonstrate that ReBoot matches the accuracy of plaintext 32-bit floating-point (FP32) plaintext training, while enabling fully non-interactive end-to-end encrypted training. Moreover, it improves test accuracy by up to +3.27% over encrypted logistic regression, and up to +6.83% over existing encrypted DNN frameworks, while reducing training latency by up to 8.83×.

The paper is organized as follows. Section 2 surveys related literature on encrypted training of ML and DL models. Section 3 introduces the fundamentals of the CKKS scheme. Section 4 details our proposed ReBoot solution, while experimental results are presented in Section 5. Lastly, conclusions and future research directions are discussed in Section 6.

2 Related Literature

This section provides an overview of HE literature on encrypted training of ML and DL models. Due to its architectural simplicity, logistic regression was among the first algorithms to be trained under encryption. Early studies presented proof-of-concept implementations using bootstrapping-free CKKS scheme [9, 24, 36]. Subsequent research extended these works by integrating approximate bootstrapping into the training procedure, enabling more practical, real-world applications [15, 32]. Building on these foundations, several studies have demonstrated the computational feasibility of encrypted fine-tuning as an effective method for applying transfer learning in privacy-critical contexts [3, 35, 37, 47, 57, 61]. Rather than retraining entire networks, these approaches focus on updating only the final fully-connected layer of a pre-trained model, making them particularly well-suited for CKKS-based computation.

¹The code will be released as a public repository in the next phase.

More recent research has shifted toward training full DNNs over encrypted data by leveraging FHE schemes. To this end, several works have employed TFHE to develop learning algorithms that redesign small Multi-Layer Perceptrons (MLPs) to operate using encrypted Boolean logic gates [23, 42, 60]. In particular, Montero et al. [42] explored interaction-based training, in which ciphertexts are *re-encrypted* to refresh noise level after each mini-batch pass. Yoo and Yoon [60] and Colombo et al. [23] leveraged encrypted Boolean logic gates to implement commonly used MLP functions, as well as cross-validation procedures. By integrating bootstrapping, both works support non-interactive training of arbitrarily deep Neural Networks (NNs). However, the lack of *SIMD* capabilities in the TFHE scheme forces homomorphic operations to be executed sequentially, resulting in prohibitive latency for practical DNN training. Consequently, these works remain at the proof-of-concept stage, demonstrating TFHE-based training only on toy-example architectures.

Other FHE schemes remain relatively underexplored. For instance, Nandakumar et al. [44] investigated the use of the Brakerski–Gentry–Vaikuntanathan (BGV) scheme [13] to train small MLPs through a non-interactive learning algorithm that leverages BGV bootstrapping to refresh ciphertexts. Additionally, Lou et al. [39] proposed a hybrid approach that combines the BFV and TFHE schemes to optimize training performance. Their method employs a BFV-based training algorithm for efficient arithmetic operations, while switching ciphertexts to the TFHE domain via *key switching* to perform non-linear functions and bootstrapping. This design leverages the strengths of both schemes, thereby improving overall training times. Nonetheless, both BGV and BFV operate over integer values, necessitating the quantization of data and model parameters. This constraint requires the use of complex integer-based learning algorithms [49], which not only increase design complexity but also degrade model accuracy compared to training in the real-valued domain.

To preserve full-precision weights during training while maintaining feasible latency, Mihara et al. [41] proposed a solution that leverages CKKS within an interaction-based protocol between the data owner (i.e., the client) and the service provider (i.e., the server). Their approach relies on a *re-encryption* mechanism to both refresh noise in the model weights and re-order values within the ciphertexts. While effective in preserving model integrity, the solution is unsuitable for *as-a-service* scenarios due to its reliance on frequent client-server interactions, which introduce significant communication overhead and operational costs.

In this context, ReBoot stands out as the first framework to enable fully encrypted non-interactive training of real-valued DNNs. ReBoot introduces both a novel encrypted architecture and a corresponding encrypted learning algorithm that leverage the proposed packing technique to exploit CKKS *SIMD* operations while addressing the scheme’s inherent limitations, thereby reducing computational and memory demands.

3 The CKKS Homomorphic Encryption Scheme

Homomorphic encryption (HE) is a class of encryption schemes that allows computations to be performed directly on encrypted data, while ensuring that the result obtained from computations on

ciphertexts matches the result of performing the same operation on the corresponding plaintexts [1]. Formally, an encryption function $E(\text{pk}, \cdot)$ and its corresponding decryption function $D(\text{sk}, \cdot)$ are considered homomorphic with respect to (w.r.t.) a set of functions $\mathcal{F}$ if, for any function $f \in \mathcal{F}$, there exists a function g such that $f(x) = D(\text{sk}, g(E(\text{pk}, x)))$ for any input x , where pk and sk denote the public and secret keys, respectively [11]. This property relies on the preservation of the data's algebraic structure throughout the encryption and computation processes [46].

Among the various HE schemes, our study employs CKKS [19]. Specifically, CKKS is based on a widely used quantum-resistant computational problem known as the Ring Learning With Errors (RLWE) problem [17, 40]. It supports approximate arithmetic over encrypted complex numbers, making it well-suited for real-valued ML and DL applications. CKKS belongs to the family of *leveled* HE schemes, which allow only a finite number of consecutive encrypted operations before the underlying plaintext becomes unrecoverable. This limit, known as *scheme level* and denoted by l , arises from noise injected into ciphertexts to ensure the probabilistic security guarantees of the RLWE problem [31]. The noise grows with each operation, eventually leading to an unrecoverable rounding error upon decryption.

The algebraic structure of CKKS plaintexts and ciphertexts is defined by a set of encryption parameters $\Theta = \{N, \mathbf{q}, \Delta\}$, where N is a power of two defining the *polynomial degree*, $\mathbf{q}$ is a list of $l + 2$ prime numbers known as the *coefficient moduli*, and Δ defines the *scaling factor*. The parameters Θ determine the scheme's *security level*, denoted by λ , as well as the encoding precision and the number of available operations. Each time a ciphertext is multiplied, its coefficient modulus is reduced by switching to the next prime in the modulus chain $\mathbf{q}$. This process continues until level l is reached, beyond which further multiplications are not permitted due to excessive noise accumulation. Consequently, the choice of scheme level l is critical, as it determines the maximum supported multiplicative depth and directly influences ciphertext size. A higher level l requires a larger polynomial degree N , which, in turn, significantly increases both computational complexity and memory consumption [5]. Given the inherently complex nature of DNN training, minimizing the required multiplicative depth is essential to ensure efficient execution under encryption.

CKKS supports *batching* [53], a technique that enables parallel processing through *Single Instruction, Multiple Data (SIMD)* operations [29]. Using batching, a vector of up to $N/2$ values can be encoded into a single ciphertext, thereby reducing both memory and computational overhead. Formally, let $\mathbf{z} \in \mathbb{C}^{N/2}$ be a vector of complex numbers to be encrypted using CKKS. Before encryption, $\mathbf{z}$ is mapped to a polynomial $z(X) \in \mathcal{R}$ using a variant of the *complex canonical embedding* map, denoted as $\phi : \mathbb{C}^{N/2} \rightarrow \mathcal{R}$, where $\mathcal{R} = \mathbb{Z}[X]/(X^N + 1)$ is the polynomial ring in which plaintexts reside. The polynomial $z(X)$ is then encrypted using the secret key sk to produce a ciphertext $\underline{z} \in \mathcal{R}_{\mathbf{q}} = \mathbb{Z}_{q_0}[X]/(X^N + 1)$. During encryption, a noise term is added to ensure the probabilistic nature of the encryption scheme, thereby preventing deterministic attacks.

The adopted CKKS scheme natively supports element-wise operations on encrypted vectors. Let $\underline{\mathbf{a}} = [\underline{a}_1, \underline{a}_2, \dots, \underline{a}_{N/2}]$ and $\underline{\mathbf{b}} =$

$[\underline{b}_1, \underline{b}_2, \dots, \underline{b}_{N/2}]$ be two encrypted CKKS vectors. The element-wise homomorphic addition, subtraction, and multiplication, denoted by $+\bullet$, $-\bullet$, and $\odot\bullet$, respectively, are defined as follows:

$$\underline{\mathbf{a}} +\bullet \underline{\mathbf{b}} = [\underline{a}_1 + \underline{b}_1, \underline{a}_2 + \underline{b}_2, \dots, \underline{a}_{N/2} + \underline{b}_{N/2}], \quad (1)$$

$$\underline{\mathbf{a}} -\bullet \underline{\mathbf{b}} = [\underline{a}_1 - \underline{b}_1, \underline{a}_2 - \underline{b}_2, \dots, \underline{a}_{N/2} - \underline{b}_{N/2}], \quad (2)$$

$$\underline{\mathbf{a}} \odot\bullet \underline{\mathbf{b}} = [\underline{a}_1 \cdot \underline{b}_1, \underline{a}_2 \cdot \underline{b}_2, \dots, \underline{a}_{N/2} \cdot \underline{b}_{N/2}]. \quad (3)$$

Similarly, element-wise addition, subtraction, and multiplication can be performed between a plaintext vector $\mathbf{z}$ and an encrypted vector $\underline{\mathbf{a}}$. In this case, the operations are denoted by $+\circ$, $-\circ$, and $\odot\circ$, respectively. The final operation supported by CKKS is homomorphic vector rotation, which enables a cyclic shift of the elements batched within a single ciphertext. A rotation by k positions of the encrypted vector $\underline{\mathbf{a}}$ results in a new encrypted vector whose elements are cyclically shifted as follows:

$$\text{Rot}_\bullet(\underline{\mathbf{a}}, k) = [\underline{a}_{k+1}, \underline{a}_{k+2}, \dots, \underline{a}_k].$$

Rotation keys are required to perform this operation, with one key corresponding to each desired rotation index k .

By leveraging batching, matrices can be encoded in their flattened form into a single ciphertext. This representation enables the definition of aggregate operations that compute homomorphic summations across specific dimensions of the original matrix [32]. Consider a flattened encrypted CKKS matrix

$$\underline{\mathbf{w}} = [\underline{w}_{1,1}, \dots, \underline{w}_{1,c}, \underline{w}_{2,1}, \dots, \underline{w}_{2,c}, \dots, \underline{w}_{r,1}, \dots, \underline{w}_{r,c}]$$

of dimension $r \times c = N/2$, where both r and c are powers of two. The summation across rows is defined as

$$\underline{\mathbf{s}} = \sum_{\bullet}^{\text{rows}} \underline{\mathbf{w}}, \quad (4)$$

where

$$\underline{\mathbf{s}} = [\underline{s}_1, \dots, \underline{s}_c, \underline{s}_1, \dots, \underline{s}_c, \dots, \underline{s}_1, \dots, \underline{s}_c]$$

denotes a flattened encrypted matrix in which each value $\underline{s}_j$, with $j \in \{1, \dots, c\}$, is replicated r times along the row dimension and represents the j -th column-wise sum of $\underline{\mathbf{w}}$. More in detail, the summation across rows is computed by recursively performing homomorphic element-wise additions and rotations. Let $\underline{\mathbf{s}}^0 = \underline{\mathbf{w}}$ be the initial ciphertext. Then, for each iteration $t \in \{0, \dots, \log_2(r)-1\}$, the partial column-wise sums are computed as

$$\underline{\mathbf{s}}^{t+1} = \underline{\mathbf{s}}^t +\bullet \text{Rot}_\bullet(\underline{\mathbf{s}}^t, c \cdot 2^{\log_2(r)-1-t}).$$

After the final iteration, the ciphertext $\underline{\mathbf{s}} = \underline{\mathbf{s}}^{\log_2(r)}$ contains the replicated column-wise sums of $\underline{\mathbf{w}}$.

Similarly, the summation across columns is defined as

$$\underline{\mathbf{s}} = \sum_{\bullet}^{\text{cols}} \underline{\mathbf{w}}, \quad (5)$$

where

$$\underline{\mathbf{s}} = [\underline{s}_1, \dots, \underline{s}_1, \underline{s}_2, \dots, \underline{s}_2, \dots, \underline{s}_r, \dots, \underline{s}_r]$$

denotes a flattened encrypted matrix in which each value $\underline{s}_i$, with $i \in \{1, \dots, r\}$, is replicated c times along the column dimension and represents the i -th row-wise sum of $\underline{\mathbf{w}}$. Specifically, the summation across columns involves three steps. First, the ciphertext accumulating the row-wise sums is computed recursively using homomorphic element-wise additions and rotations. Let $\underline{\mathbf{u}}^0 = \underline{\mathbf{w}}$ denote the initial

ciphertext. Then, for each iteration $t \in \{0, \dots, \log_2(c) - 1\}$, the partial row-wise sums are computed as

$$\underline{\mathbf{u}}^{t+1} = \underline{\mathbf{u}}^t + \bullet \text{Rot}_\bullet \left(\underline{\mathbf{u}}^t, 2^{\log_2(c)-1-t} \right).$$

After the final iteration, the ciphertext

$$\underline{\mathbf{u}}^{\log_2(c)} = [\underline{u}_1, \perp, \dots, \perp, \underline{u}_2, \perp, \dots, \perp, \dots, \underline{u}_r, \perp, \dots, \perp]$$

contains the i -th row-wise sum in the first slot of each row i , while the remaining $c - 1$ slots in each row contain corrupted values, denoted by $\perp$. These corrupted values result from additions with elements from subsequent rows introduced by rotation operations. Second, to retain only the values $\underline{u}_i$ in the first slot of each row within the ciphertext $\underline{\mathbf{u}}^{\log_2(c)}$, with $i \in \{1, \dots, r\}$, a masking operation is applied:

$$\underline{\mathbf{u}} = \mathbf{m} \odot \underline{\mathbf{u}}^{\log_2(c)},$$

where $\mathbf{m} \in \{0, 1\}^{N/2}$ is a flattened binary mask containing all zeros except for the first slot of each row, which is set to 1. It is worth noting that, unlike the summation across rows, the summation across columns requires a multiplicative depth of 1 due to this masking step. Third, to obtain the final ciphertext $\underline{\mathbf{s}}$, the row-wise sums are replicated c times along the column dimension by recursively performing homomorphic element-wise additions and rotations. Let $\underline{\mathbf{s}}^0 = \underline{\mathbf{u}}$ be the initial ciphertext. Then, for each iteration $t \in \{0, \dots, \log_2(c) - 1\}$, the partially replicated sums are computed as

$$\underline{\mathbf{s}}^{t+1} = \underline{\mathbf{s}}^t + \bullet \text{Rot}_\bullet \left(\underline{\mathbf{s}}^t, -2^t \right).$$

After the final iteration, the ciphertext $\underline{\mathbf{s}} = \underline{\mathbf{s}}^{\log_2(c)}$ contains the replicated row-wise sums of $\underline{\mathbf{w}}$.

To overcome the limitation on the number of consecutive multiplications and enable an arbitrary number of homomorphic operations, CKKS supports a procedure known as *bootstrapping* [30]. Bootstrapping refreshes the ciphertext by homomorphically restoring its modulus level, thereby allowing additional homomorphic operations beyond the original scheme level l . Formally, the goal of the bootstrapping operation applied to a ciphertext $\underline{\mathbf{c}}$ at level $l = 0$ is to compute a refreshed ciphertext $\underline{\mathbf{c}}' = \text{Bootstrap}(\underline{\mathbf{c}})$ at level $l' - d > 0$, where d is the multiplicative depth of the bootstrapping circuit, such that $\underline{\mathbf{c}}' \approx \underline{\mathbf{c}}$. Cheon et al. [18] proposed the first procedure to compute this modular reduction in CKKS by homomorphically applying the encoding step, approximating the modular reduction via a scaled sine function, and then applying the decoding step². However, unlike other FHE schemes such as BGV [13] and TFHE [20], CKKS bootstrapping does not refresh the noise accumulated in the ciphertext, due to the scheme's approximate nature [26]. As a result, even with bootstrapping, it remains impossible to reduce the noise in CKKS ciphertexts, requiring us to rethink and redesign both standard DNN architectures and training algorithms to natively take into account these constraints.

4 The Proposed Solution

This section introduces the proposed ReBoot framework, specifically designed for HE-compliant encrypted training and inference

²Refer to Bossuat et al. [12] and Bae et al. [8] for an in-depth description of CKKS bootstrapping as used in this work.

of MLPs. Section 4.1 presents the packing schemes designed to efficiently encode inputs, weights, activations, and gradients. The encrypted NN architecture is detailed in Section 4.2, while Section 4.3 describes the encrypted learning algorithm. Finally, Section 4.4 provides a comprehensive analysis of the multiplicative depth required by the ReBoot framework.

4.1 Reboot Packing

Reboot encrypted processing is designed to maximize computational efficiency by employing a structured packing strategy within the ciphertext space. By carefully organizing data inside each ciphertext, our approach enables highly efficient encrypted computation, significantly enhancing the training of MLPs on encrypted data.

ReBoot employs two distinct packing schemes to encode vectors and matrices into ciphertexts. In particular, inputs, weights, activations, and gradients are packed into matrices $\tilde{\mathbf{M}} \in \mathbb{R}^{r \times c}$, with $r \times c = N/2$, and then encrypted into single ciphertexts $\underline{\mathbf{m}}$. The encoding dimensions, namely r and c , are determined by the DNN architecture, as explained in Section 4.2.3, and remain consistent across all packed matrices. This packing structure is essential for defining *a priori* a fixed set of CKKS rotation keys, which are required for ciphertext rotations, as explained in Section 3. Furthermore, it enables full exploitation of *SIMD* operations on ciphertexts. By ensuring consistent and compact data representation, ReBoot packing reduces the number of ciphertext and minimizes the required homomorphic rotations and multiplications, thereby significantly lowering memory usage and computational overhead during encrypted model training.

Let $\mathbf{a} = [a_1, \dots, a_d] \in \mathbb{R}^d$ be a one-dimensional vector, with $d \leq N/2$. Reboot packing encodes the input vector $\mathbf{a}$ into a matrix $\tilde{\mathbf{A}} \in \mathbb{R}^{r \times c}$. Specifically, the matrix $\tilde{\mathbf{A}}$ can be structured in the *repeated* format

$$\tilde{\mathbf{A}}_{\text{rep}} = \begin{bmatrix} a_1 & a_2 & \dots & a_d & 0 & \dots & 0 \\ a_1 & a_2 & \dots & a_d & 0 & \dots & 0 \\ \vdots & \vdots & \dots & \vdots & \vdots & \dots & \vdots \\ a_1 & a_2 & \dots & a_d & 0 & \dots & 0 \end{bmatrix},$$

where each row $i \in \{1, \dots, r\}$ contains a copy of the vector $\mathbf{a}$, zero-padded to match the row length c . Alternatively, it can be structured in the *expanded* format

$$\tilde{\mathbf{A}}_{\text{exp}} = \begin{bmatrix} a_1 & a_1 & \dots & a_1 \\ a_2 & a_2 & \dots & a_2 \\ \vdots & \vdots & \dots & \vdots \\ a_d & a_d & \dots & a_d \\ 0 & 0 & \dots & 0 \\ \vdots & \vdots & \dots & \vdots \\ 0 & 0 & \dots & 0 \end{bmatrix},$$

where each column $j \in \{1, \dots, c\}$ contains a copy of the vector $\mathbf{a}$, zero-padded to match the column length r . Both representations are padded with zeros to conform to the matrix dimensions $r \times c$.

Similarly, let

$$\mathbf{W} = \begin{bmatrix} W_{11} & W_{12} & \cdots & W_{1k} \\ W_{21} & W_{22} & \cdots & W_{2k} \\ \vdots & \vdots & \ddots & \vdots \\ W_{d1} & W_{d2} & \cdots & W_{dk} \end{bmatrix} \in \mathbb{R}^{d \times k}$$

be a two-dimensional matrix, with $d \times k \leq N/2$. Reboot packing encodes the input matrix $\mathbf{W}$ into a matrix $\tilde{\mathbf{W}} \in \mathbb{R}^{r \times c}$. Specifically, the matrix $\tilde{\mathbf{W}}$ can be structured in the *row-wise* encoded format

$$\tilde{\mathbf{W}}_{\text{row}} = \begin{bmatrix} W_{11} & W_{12} & \cdots & W_{1k} & 0 & \cdots & 0 \\ W_{21} & W_{22} & \cdots & W_{2k} & 0 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ W_{d1} & W_{d2} & \cdots & W_{dk} & 0 & \cdots & 0 \\ 0 & 0 & \cdots & 0 & 0 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 0 & 0 & \cdots & 0 \end{bmatrix},$$

where the input matrix $\mathbf{W}$ is padded with $r - d$ rows and $c - k$ columns of zeros. Alternatively, the matrix $\tilde{\mathbf{W}}$ can be structured in the *column-wise* encoded format

$$\tilde{\mathbf{W}}_{\text{col}} = \begin{bmatrix} W_{11} & W_{21} & \cdots & W_{d1} & 0 & \cdots & 0 \\ W_{12} & W_{22} & \cdots & W_{d2} & 0 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ W_{1k} & W_{2k} & \cdots & W_{dk} & 0 & \cdots & 0 \\ 0 & 0 & \cdots & 0 & 0 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 0 & 0 & \cdots & 0 \end{bmatrix},$$

where the transposed matrix $\mathbf{W}^\top$ is padded with $r - k$ rows and $c - d$ columns of zeros. Note that ReBoot packing employs zero-padding to maintain consistent encoding dimensions across ciphertexts.

The proposed packing strategy ensures full utilization of ciphertext slots, which is critical to maximize CKKS *SIMD* capabilities during encrypted MLP training and inference. By exploiting these packing characteristics, ReBoot enables encrypted vector-matrix multiplications using only element-wise operations and eliminates the need for *repacking*, a common procedure used to reorder values within ciphertexts. This approach minimizes both the number of homomorphic operations and the required multiplicative depth, thereby reducing the frequency of costly bootstrapping operations.

4.2 ReBoot Encrypted Architecture

This section presents the encrypted NN architecture of ReBoot, illustrated in Figure 1, along with its main components. Specifically, Section 4.2.1 introduces the encrypted fully-connected layer. The polynomial approximation used for the encrypted activation function is described in Section 4.2.2. Finally, Section 4.2.3 details the encrypted local-loss block, where layers are stacked together.

The proposed ReBoot framework is designed for training encrypted MLPs on multi-class classification tasks defined over a dataset $\mathbf{X} = \{\mathbf{x}^\mu, \mathbf{y}^\mu\}_{\mu=1}^Q$, where $\mathbf{x}^\mu \in \mathbb{R}^{k_0}$ are the input samples of dimension k_0 , $\mathbf{y}^\mu \in \{1, \dots, o\}$ their corresponding class labels (with o denoting the number of classes), and Q is the size of the training set. Before training, each sample $\mathbf{x}^\mu$ and its corresponding

one-hot class label $\mathbf{y}^\mu$ are first encoded using the proposed packing techniques into $\tilde{\mathbf{X}}^\mu$ and $\tilde{\mathbf{Y}}^\mu$, then flattened and encrypted into ciphertexts $\tilde{\mathbf{x}}^\mu$ and $\tilde{\mathbf{y}}^\mu$, respectively.

4.2.1 Encrypted Fully-Connected Layer. ReBoot encrypted fully-connected layer is designed to compute vector-matrix multiplication between an encrypted activation vector $\tilde{\mathbf{a}}$ and an encrypted weight matrix $\tilde{\mathbf{w}}$, resulting in an encrypted pre-activation vector $\tilde{\mathbf{z}}$. Specifically, it leverages ReBoot packing schemes, as detailed in the previous section, to efficiently perform *SIMD* computations. The proposed encrypted vector-matrix multiplication can take two forms, depending on the dimension along which the summation is performed: (i) Row Encrypted Matrix Multiplication (*RE-Matmul*), and (ii) Column Encrypted Matrix Multiplication (*CE-Matmul*). For the ease of meaning, and without loss of generality, we will henceforth consider the non-flattened packed representations of the ciphertexts $\tilde{\mathbf{a}}$, $\tilde{\mathbf{w}}$, and $\tilde{\mathbf{z}}$, denoted as $\tilde{\mathbf{A}}$, $\tilde{\mathbf{W}}$, and $\tilde{\mathbf{Z}}$, respectively.

The *RE-Matmul* operation implements the summation across rows $\Sigma_{\bullet}^{\text{rows}}$, as defined in Equation 4, and is expressed as

$$\tilde{\mathbf{Z}}_{\text{rep}} = \text{RE-Matmul}(\tilde{\mathbf{A}}_{\text{exp}}, \tilde{\mathbf{W}}) = \Sigma_{\bullet}^{\text{rows}} \tilde{\mathbf{A}}_{\text{exp}} \odot_{\bullet} \tilde{\mathbf{W}}, \quad (6)$$

where $\odot_{\bullet}$ denotes homomorphic element-wise multiplication defined in Equation 3, $\tilde{\mathbf{A}}_{\text{exp}}$ and $\tilde{\mathbf{Z}}_{\text{rep}}$ represent the encrypted activation and pre-activation vectors in the *expanded* and *repeated* formats, respectively, and $\tilde{\mathbf{W}}$ is the encrypted weight matrix. As detailed in Section 3, summation across rows replicates each column-wise sum r times along the row dimension, effectively resulting in the *repeated* format. From a computational perspective, *RE-Matmul* requires a multiplicative depth of 1, contributed by the element-wise multiplication, along with $\log_2(r)$ additions and rotations. Conversely, *CE-Matmul* implements the summation across columns $\Sigma_{\bullet}^{\text{cols}}$, as defined in Equation 5, and is expressed as

$$\tilde{\mathbf{Z}}_{\text{exp}} = \text{CE-Matmul}(\tilde{\mathbf{A}}_{\text{rep}}, \tilde{\mathbf{W}}) = \Sigma_{\bullet}^{\text{cols}} \tilde{\mathbf{A}}_{\text{rep}} \odot_{\bullet} \tilde{\mathbf{W}}, \quad (7)$$

where $\tilde{\mathbf{A}}_{\text{rep}}$ and $\tilde{\mathbf{Z}}_{\text{exp}}$ denote the encrypted activation and pre-activation vectors in the *repeated* and *expanded* formats, respectively. Summation across columns replicates each row-wise sum c times along the column dimension, effectively resulting in the *expanded* format. *CE-Matmul* requires a multiplicative depth of 2, contributed by the element-wise multiplication and the masking step in the summation across columns operation, as explained in Section 3, along with $2 \log_2(c)$ additions and rotations.

Depending on the encoding of its weight matrix, ReBoot encrypted fully-connected layer can be categorized into two types: (i) the Row Encrypted Fully-Connected layer (*RE-FC*), whose weight matrix is in the *row-wise* encoded format $\tilde{\mathbf{W}}_{\text{row}}$, and (ii) the Column Encrypted Fully-Connected layer (*CE-FC*), whose weight matrix is in the *column-wise* encoded format $\tilde{\mathbf{W}}_{\text{col}}$. As explained in Section 4.3.1, during the forward pass, the *RE-FC* layer performs a *RE-Matmul* operation between the *expanded* activation vector $\tilde{\mathbf{A}}_{\text{exp}}$ and the *row-wise* encoded weight matrix $\tilde{\mathbf{W}}_{\text{row}}$. Conversely, the *CE-FC* layer performs a *CE-Matmul* operation between the *repeated* activation vector $\tilde{\mathbf{A}}_{\text{rep}}$ and the *column-wise* encoded weight matrix $\tilde{\mathbf{W}}_{\text{col}}$. As a result, the output of a *RE-FC* layer is a pre-activation vector in the *repeated* format, which aligns exactly with the input

format expected by the *CE-FC* layer, and vice versa. This alternation between output and input formats enables the composition of consecutive encrypted layers without requiring explicit repacking of intermediate ciphertexts, as detailed in Section 4.2.3. Thanks to *RE-FC* and *CE-FC* layers, ReBoot can implement efficient and scalable inference and training of MLPs.

Let $\mathcal{H} = \{1, \dots, H\}$ denote the set of ReBoot encrypted fully-connected layers in the considered MLP, where H is the total number of hidden layers. Let $k_h \in \mathbb{N}$ denote the dimensionality of layer $h \in \mathcal{H}$, with k_0 representing the input dimension of the encrypted MLP. The set $\mathcal{H}$ can be partitioned into two disjoint subsets: (i) the set of *RE-FC* layers, defined as $\mathcal{R} = \{h \in \mathcal{H} \mid h \bmod 2 = 1\}$, corresponding to odd-indexed layers, and (ii) the set of *CE-FC* layers, defined as $\mathcal{C} = \{h \in \mathcal{H} \mid h \bmod 2 = 0\}$, corresponding to even-indexed layers. The encoding dimensions of inputs, weights, activations, and gradients used in ReBoot packing, namely r and c , as explained in Section 4.1, are defined as follows:

$$\begin{cases} r = 2^{\lceil \log_2(\max(k_0, \max_{h \in \mathcal{R}} k_h)) \rceil} \\ c = 2^{\lceil \log_2(\max_{h \in \mathcal{C}} k_h) \rceil} \end{cases},$$

where $\lceil \cdot \rceil$ denotes the ceiling operation.

4.2.2 Encrypted Activation Function. ReBoot encrypted activation function is designed to apply an encrypted non-linearity to the encrypted pre-activation vector $\tilde{\mathbf{Z}}$, producing the encrypted activation vector $\tilde{\mathbf{A}}$ as output. However, due to the limited set of operations supported by CKKS, commonly used activation functions in MLP models, such as ReLU, tanh, and sigmoid, cannot be directly applied in the encrypted domain. To address this limitation, ReBoot leverages *EncryptedPolyReLU*, a second-degree polynomial approximation of the *ReLU* function, specifically designed to be HE-compliant [2, 6]. *EncryptedPolyReLU* is defined as follows:

$$\tilde{\mathbf{A}} = (\tilde{\mathbf{Z}} \odot \tilde{\mathbf{Z}}) + \bullet \tilde{\mathbf{Z}}, \quad (8)$$

where $+$ and $\odot$ denote homomorphic element-wise addition and multiplication, respectively, as defined in Equation 1 and Equation 3. From a computational perspective, *EncryptedPolyReLU* has a multiplicative depth of 1, contributed by the element-wise multiplication, along with a single addition. Since only element-wise operations are involved, *EncryptedPolyReLU* does not alter the encoding structure between input and output encrypted vectors. Accordingly, if the input is in the *expanded* format, the output will also be in the *expanded* format, and vice versa.

4.2.3 Encrypted Local-Loss Block. To minimize multiplicative depth required for encrypted training, and consequently the frequency of costly bootstrapping operations, ReBoot encrypted architecture segments the MLP into multiple encrypted local-loss blocks. This design aligns with a stream of literature that employs local layer-wise loss functions [43, 45, 48]. The key advantage of ReBoot architecture lies in its ability to confine gradient propagation within each encrypted local-loss block, thereby eliminating the need to back-propagate gradients through the entire NN. As a result, encrypted local-loss blocks restrict HE noise accumulation during training. Moreover, they enable the independent training of each block, since no information flows between blocks during the backward pass.

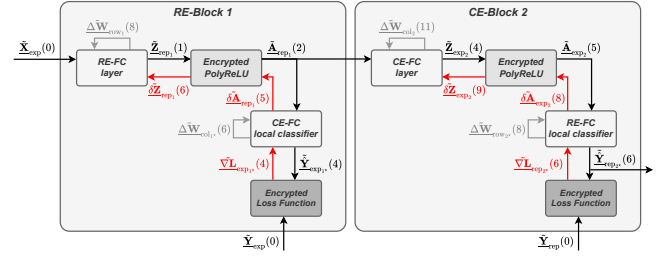


Figure 1: Overview of ReBoot encrypted architecture for a MLP with two hidden layers. The contributions of the forward pass, backward pass, and weight updates are shown in black, red, and gray, respectively. The multiplicative depth of each value is indicated in parentheses.

This property allows for the parallelization of the training procedure, significantly reducing computational overhead. A detailed analysis of ReBoot’s multiplicative depth is provided in Section 4.4.

ReBoot encrypted architecture is illustrated in Figure 1. Each ReBoot encrypted fully-connected layer $h \in \mathcal{H}$, along with its associated *EncryptedPolyReLU*, is paired with a local classifier h^* of size $k_h \times o$, forming an encrypted local-loss block B_h . The local classifier h^* , which is itself a ReBoot encrypted fully-connected layer, serves to reduce the dimensionality of the encrypted activation vector $\tilde{\mathbf{A}}_h$ to the output dimension o . During the backward pass, the local output $\tilde{\mathbf{Y}}_h$ and the true label $\tilde{\mathbf{Y}}$ are used by the local loss function L_h to compute the local gradient $\tilde{\mathbf{V}}\mathbf{L}_h$. This gradient is then used by ReBoot training algorithm, detailed in Section 4.3, to update the weights of both the encrypted fully-connected layer h and its local classifier h^* . In the final block B_H , the local classifier H^* serves as the NN’s output layer, producing the final prediction $\tilde{\mathbf{Y}}'_H$. Once the training process is completed, the intermediate local classifiers can either be discarded or leveraged for early-exit strategies [16, 51, 55].

Encrypted local blocks B_h can be of two types: *Row Encrypted Blocks (RE-Blocks)*, when $h \in \mathcal{R}$, and *Column Encrypted Blocks (CE-Blocks)*, when $h \in \mathcal{C}$. It is worth noting that a *RE-Block* has a local classifier $h^* \in \mathcal{C}$, whereas a *CE-Block* has a local classifier $h^* \in \mathcal{R}$. By structuring ReBoot encrypted architecture to alternate between *RE-Blocks* and *CE-Blocks*, compatibility between encrypted fully-connected layers is inherently preserved, eliminating the need for auxiliary transformations. This property offers a significant computational advantage, as homomorphic repacking operations are typically expensive in terms of multiplicative depth, a critical factor for minimizing noise growth and avoiding costly bootstrapping operations. Consequently, alternating between *RE-Blocks* and *CE-Blocks* not only preserves packing efficiency across layers but also enables modular and scalable encrypted model architectures, making ReBoot particularly well-suited for practical encrypted MLP training and inference.

4.3 ReBoot Encrypted Learning Algorithm

This section outlines the key components of ReBoot encrypted learning algorithm, as detailed in Algorithm 1 and Algorithm 2. Since each encrypted local-loss block is trained independently, as

explained in Section 4.2.3, and the extension to *CE-Blocks* is straightforward, we focus on describing the training procedure for a representative *RE-Block* B_h , with $h \in \mathcal{R}$. Specifically, Section 4.3.1 describes the forward pass of a *RE-Block*. Section 4.3.2 introduces the encrypted loss function along with its gradient computation. In Section 4.3.3, the backward pass within the *RE-Block* is presented. Finally, Section 4.3.4 details the encrypted weight update process.

4.3.1 Forward Pass. The forward pass procedure is executed at the beginning of each iteration $t \in \{1, \dots, T\}$, where T denotes the total number of training iterations. Let $\tilde{\mathbf{A}}_{\text{exp}_{h-1}}^t$ be the encrypted activations produced by the previous *CE-Block* B_{h-1} . Notably, in the first local-loss block (i.e., when $h = 1$), the input activations correspond to the input training samples, i.e., $\tilde{\mathbf{A}}_{\text{exp}_0}^t = \tilde{\mathbf{X}}_{\text{exp}}^t$. The *RE-Block* computes the encrypted pre-activations as

$$\tilde{\mathbf{Z}}_{\text{rep}_h}^t = \text{RE-Matmul}(\tilde{\mathbf{A}}_{\text{exp}_{h-1}}^t, \tilde{\mathbf{W}}_{\text{row}_h}^t),$$

where $\tilde{\mathbf{W}}_{\text{row}_h}^t$ denotes the encrypted weights of the *RE-FC* layer h in the *row-wise* encoded format, and *RE-MatMul* refers to the row encrypted matrix multiplication described in Equation 6. The resulting encrypted pre-activations $\tilde{\mathbf{Z}}_{\text{rep}_h}^t$ are then passed through the *EncryptedPolyReLU* activation function described in Equation 8, yielding the encrypted activations $\tilde{\mathbf{A}}_{\text{rep}_h}^t$. These activations are simultaneously propagated to both the next *CE-Block* B_{h+1} and the corresponding local classifier h^* . Specifically, the local classifier produces the encrypted predictions

$$\tilde{\mathbf{Y}}_{\text{exp}_h^*}^t = \text{CE-Matmul}(\tilde{\mathbf{A}}_{\text{rep}_h}^t, \tilde{\mathbf{W}}_{\text{col}_h^*}^t),$$

where $\tilde{\mathbf{W}}_{\text{col}_h^*}^t$ denotes the encrypted weights of the *CE-FC* local classifier h^* in the *column-wise* encoded format, and *CE-Matmul* refers to the column encrypted matrix multiplication defined in Equation 7. The local predictions $\tilde{\mathbf{Y}}_{\text{exp}_h^*}^t$ are then used by the encrypted loss function, described in the following section, to compute the gradients required to update the weights of both the *RE-FC* layer h and the *CE-FC* local classifier h^* . This design mitigates ciphertext noise accumulation by limiting multiplicative depth, as analyzed in Section 4.4.

4.3.2 Encrypted Loss Function. The objective of ReBoot encrypted learning algorithm is to minimize the encrypted local loss function L_h at each local block B_h . Specifically, ReBoot employs the *Residual Sum of Squares (RSS)* loss, defined as

$$\tilde{L}_{\text{exp}_h^*}^t = (\tilde{\mathbf{Y}}_{\text{exp}_h^*}^t - \tilde{\mathbf{Y}}_{\text{exp}_h^*}^t) \odot (\tilde{\mathbf{Y}}_{\text{exp}_h^*}^t - \tilde{\mathbf{Y}}_{\text{exp}_h^*}^t),$$

where $\tilde{\mathbf{Y}}_{\text{exp}_h^*}^t$ denotes the encrypted predictions and $\tilde{\mathbf{Y}}_{\text{exp}_h^*}^t$ the encrypted one-hot target labels. Here, $-$ and $\odot$ represent homomorphic element-wise subtraction and multiplication, as defined in Equation 2 and Equation 3, respectively.

The encrypted gradient of the loss w.r.t. the predictions, used to update the weights during the backward pass, is given by

$$\tilde{\nabla L}_{\text{exp}_h^*}^t = \tilde{\mathbf{Y}}_{\text{exp}_h^*}^t - \tilde{\mathbf{Y}}_{\text{exp}_h^*}^t.$$

The *RSS* gradient offers two key advantages in the encrypted domain. First, it does not increase the multiplicative depth, thereby

Algorithm 1 ReBoot encrypted learning algorithm: *RE-Block* B_h

Input: $\tilde{\mathbf{W}}_{\text{row}_h}^t$: encrypted layer weights at iteration t , $\tilde{\mathbf{W}}_{\text{col}_h^*}^t$: encrypted local classifier weights at iteration t , $\tilde{\mathbf{A}}_{\text{exp}_{h-1}}^t$: encrypted activations of block B_{h-1} at iteration t , γ : learning rate, η : decay rate, μ : momentum, t : iteration

Output: $\tilde{\mathbf{W}}_{\text{row}_h}^{t+1}$: encrypted layer weights at iteration $t + 1$, $\tilde{\mathbf{W}}_{\text{col}_h^*}^{t+1}$: encrypted local classifier weights at iteration $t + 1$

// Forward Pass

$$1: \tilde{\mathbf{Z}}_{\text{rep}_h}^t \leftarrow \text{RE-Matmul}(\tilde{\mathbf{A}}_{\text{exp}_{h-1}}^t, \tilde{\mathbf{W}}_{\text{row}_h}^t)$$

$$2: \tilde{\mathbf{A}}_{\text{rep}_h}^t \leftarrow \text{EncryptedPolyReLU}(\tilde{\mathbf{Z}}_{\text{rep}_h}^t)$$

$$3: \tilde{\mathbf{Y}}_{\text{exp}_h}^t \leftarrow \text{CE-Matmul}(\tilde{\mathbf{A}}_{\text{rep}_h}^t, \tilde{\mathbf{W}}_{\text{col}_h^*}^t)$$

// Encrypted Gradient of Loss Function

$$4: \tilde{\nabla L}_{\text{exp}_h}^t \leftarrow \tilde{\mathbf{Y}}_{\text{exp}_h}^t - \tilde{\mathbf{Y}}_{\text{exp}_h}^t$$

// Backward Pass

$$5: \tilde{\delta \mathbf{W}}_{\text{col}_h^*}^t \leftarrow \tilde{\mathbf{A}}_{\text{rep}_h}^t \odot \tilde{\nabla L}_{\text{exp}_h}^t$$

$$6: \tilde{\delta \mathbf{A}}_{\text{rep}_h}^t \leftarrow \text{RE-Matmul}(\tilde{\nabla L}_{\text{exp}_h}^t, \tilde{\mathbf{W}}_{\text{col}_h^*}^t)$$

$$7: \tilde{\delta \mathbf{Z}}_{\text{rep}_h}^t \leftarrow \text{EncryptedPolyReLU}'(\tilde{\delta \mathbf{A}}_{\text{rep}_h}^t)$$

$$8: \tilde{\delta \mathbf{W}}_{\text{row}_h}^t \leftarrow \tilde{\mathbf{A}}_{\text{exp}_{h-1}}^t \odot \tilde{\delta \mathbf{Z}}_{\text{rep}_h}^t$$

// Weight Updates

$$9: \tilde{\mathbf{W}}_{\text{row}_h}^{t+1}, \tilde{\mathbf{V}}_{\text{row}_h}^{t+1} \leftarrow \text{UpdateWeights}(\tilde{\mathbf{W}}_{\text{row}_h}^t, \tilde{\mathbf{V}}_{\text{row}_h}^t, \tilde{\delta \mathbf{W}}_{\text{row}_h}^t, \gamma, \eta, \mu, t)$$

$$10: \tilde{\mathbf{W}}_{\text{col}_h^*}^{t+1}, \tilde{\mathbf{V}}_{\text{col}_h^*}^{t+1} \leftarrow \text{UpdateWeights}(\tilde{\mathbf{W}}_{\text{col}_h^*}^t, \tilde{\mathbf{V}}_{\text{col}_h^*}^t, \tilde{\delta \mathbf{W}}_{\text{col}_h^*}^{t-1}, \gamma, \eta, \mu, t)$$

$$11: \{\tilde{\mathbf{W}}_{\text{row}_h}^{t+1}, \tilde{\mathbf{V}}_{\text{row}_h}^{t+1}\} \leftarrow \text{Bootstrap}(\{\tilde{\mathbf{W}}_{\text{row}_h}^{t+1}, \tilde{\mathbf{V}}_{\text{row}_h}^{t+1}\})$$

$$12: \{\tilde{\mathbf{W}}_{\text{col}_h^*}^{t+1}, \tilde{\mathbf{V}}_{\text{col}_h^*}^{t+1}\} \leftarrow \text{Bootstrap}(\{\tilde{\mathbf{W}}_{\text{col}_h^*}^{t+1}, \tilde{\mathbf{V}}_{\text{col}_h^*}^{t+1}\})$$

$$13: \text{return } \tilde{\mathbf{W}}_{\text{row}_h}^{t+1}, \tilde{\mathbf{W}}_{\text{col}_h^*}^{t+1}$$

limiting noise accumulation and improving runtime efficiency. Second, it coincides with the gradient of the *cross-entropy* loss when coupled with a *sigmoid* activation, allowing the model to retain meaningful learning dynamics while remaining compatible with CKKS constraints.

4.3.3 Backward Pass. Once the encrypted gradient of the loss w.r.t. the predictions has been computed, the corresponding encrypted gradients w.r.t. weights, activations, and pre-activations can be derived. First, the encrypted gradient w.r.t. the weights of the *CE-FC* local classifier $\tilde{\delta \mathbf{W}}_{\text{col}_h^*}^t$ is computed as the outer product between the input activations to the local classifier $\tilde{\mathbf{A}}_{\text{rep}_h}^t$ and the loss gradient w.r.t. the predictions $\tilde{\nabla L}_{\text{exp}_h^*}^t$. In the encrypted domain, thanks to ReBoot packing, this outer product is mapped to an element-wise multiplication $\odot$ between two vectors encoded in the *repeated* and *expanded* formats, respectively. Formally,

$$\tilde{\delta \mathbf{W}}_{\text{col}_h^*}^t = \tilde{\mathbf{A}}_{\text{rep}_h}^t \odot \tilde{\nabla L}_{\text{exp}_h^*}^t.$$

Second, the encrypted gradient w.r.t. the activations $\tilde{\delta \mathbf{A}}_{\text{rep}_h}^t$ is computed as the matrix multiplication between the loss gradient w.r.t. the predictions $\tilde{\nabla L}_{\text{exp}_h^*}^t$ and the transposed weight

matrix $\tilde{\mathbf{W}}_{\text{col}_h}^t$. Thanks to ReBoot packing, this operation can be efficiently performed using the row encrypted matrix multiplication *RE-Matmul*, as defined in Equation 6, provided that the weights are encoded in the *column-wise* format. Indeed, as explained in Section 4.1, the *column-wise* encoding inherently represents the transposed weight matrix. Consequently, no additional operations are required to reorder values within the ciphertext. Formally,

$$\delta \tilde{\mathbf{A}}_{\text{rep}_h}^t = \text{RE-Matmul}(\tilde{\mathbf{L}}_{\text{exp}_h}^t, \tilde{\mathbf{W}}_{\text{col}_h}^t).$$

Third, the encrypted gradient w.r.t. the pre-activations $\delta \tilde{\mathbf{Z}}_{\text{rep}_h}^t$ is obtained by evaluating the homomorphic derivative of the *EncryptedPolyReLU* activation function, defined in Equation 8, at the encrypted activations $\delta \tilde{\mathbf{A}}_{\text{rep}_h}^t$. Specifically, the derivative *EncryptedPolyReLU'* is computed as

$$\delta \tilde{\mathbf{Z}}_{\text{rep}_h}^t = \mathbf{1}_{r \times c} +_o \mathbf{2}_{r \times c} \odot_o \delta \tilde{\mathbf{A}}_{\text{rep}_h}^t,$$

where $\{\mathbf{1}_{r \times c}, \mathbf{2}_{r \times c}\} \in \mathbb{R}^{r \times c}$ are plaintext matrices filled with the constants 1 and 2, respectively, to match the dimensions of the encrypted activations. Here, $+_o$ and $\odot_o$ represent plaintext-ciphertext element-wise addition and multiplication, respectively.

Lastly, the encrypted gradient w.r.t. the weights of the *RE-FC* layer $\delta \tilde{\mathbf{W}}_{\text{row}_h}^t$ is obtained as

$$\delta \tilde{\mathbf{W}}_{\text{row}_h}^t = \tilde{\mathbf{A}}_{\text{exp}_{h-1}}^t \odot_o \delta \tilde{\mathbf{Z}}_{\text{rep}_h}^t.$$

At this point, ReBoot encrypted weight update algorithm is executed to update the weights of both the *CE-FC* local classifier h^* and the *RE-FC* layer h , as explained in the following section.

4.3.4 Encrypted Weight Update. Once the encrypted gradients of the loss w.r.t. the weights of the *CE-FC* local classifier h^* and the *RE-FC* layer h have been computed, they are used by ReBoot encrypted weight update algorithm, outlined in Algorithm 2, to update the corresponding weights. Given the incompatibility of most commonly used adaptive learning rate algorithms with HE, our method draws inspiration from *Nesterov Accelerated Gradient (NAG)* [54] to support both *weight decay* and *momentum* within the encrypted domain. This choice is motivated by the importance of these techniques in modulating convergence speed, which is especially important in HE-based training, where each homomorphic operation introduces additional noise and computational overhead.

Since the update procedure is general and applies to both *CE-FC* and *RE-FC* layers, we generically denote the encrypted gradient w.r.t. the weights as $\delta \tilde{\mathbf{W}}^t$. Let η denote the *weight decay rate*, μ the *momentum*, and γ the *learning rate*. The encrypted decay-regularized gradient is computed as

$$\delta \tilde{\mathbf{W}}^t = \delta \tilde{\mathbf{W}}^t +_\bullet (\eta_{r \times c} \odot_o \delta \tilde{\mathbf{W}}^t),$$

where $\eta_{r \times c} \in \mathbb{R}^{r \times c}$ is a plaintext matrix filled with the constant η to match the dimensions of the encrypted weight gradient. Here, $+\bullet$ and $\odot_o$ represent homomorphic element-wise addition and plaintext-ciphertext element-wise multiplication, respectively.

The encrypted *velocity* accumulates gradient information over time, serving as a momentum term that smooths updates and accelerates convergence. At the first iteration (i.e., $t = 1$), the encrypted

Algorithm 2 *UpdateWeights*: ReBoot encrypted weight update

Input: $\tilde{\mathbf{W}}^t$: encrypted layer weights at iteration t , $\tilde{\mathbf{V}}^t$: encrypted velocity at iteration t , $\delta \tilde{\mathbf{W}}^t$: encrypted weights gradient at iteration t , γ : learning rate, η : decay rate, μ : momentum, t : iteration

Output: $\tilde{\mathbf{W}}^{t+1}$: encrypted layer weights at iteration $t + 1$, $\tilde{\mathbf{V}}^{t+1}$: encrypted velocity at iteration $t + 1$

// Decay Regularization

1: $\delta \tilde{\mathbf{W}}^t = \delta \tilde{\mathbf{W}}^t +_\bullet (\eta_{r \times c} \odot_o \delta \tilde{\mathbf{W}}^t)$

// NAG Algorithm

2: **if** $t = 1$ **then**

3: $\tilde{\mathbf{V}}^2 \leftarrow \delta \tilde{\mathbf{W}}^1$

4: $\Delta \tilde{\mathbf{W}}^1 = (\gamma_{r \times c} \odot_o \delta \tilde{\mathbf{W}}^1) +_\bullet (\gamma \mu_{r \times c} \odot_o \delta \tilde{\mathbf{W}}^1)$

5: **else**

6: $\tilde{\mathbf{V}}^{t+1} = (\mu_{r \times c} \odot_o \tilde{\mathbf{V}}^t) +_\bullet \delta \tilde{\mathbf{W}}^t$

7: $\Delta \tilde{\mathbf{W}}^t = (\gamma_{r \times c} \odot_o \delta \tilde{\mathbf{W}}^t) +_\bullet (\gamma \mu_{r \times c} \odot_o \delta \tilde{\mathbf{W}}^t) +_\bullet (\gamma \mu^2_{r \times c} \odot_o \tilde{\mathbf{V}}^t)$

8: **end if**

9: $\tilde{\mathbf{W}}^{t+1} \leftarrow \tilde{\mathbf{W}}^t -_\bullet \Delta \tilde{\mathbf{W}}^t$

10: **return** $\tilde{\mathbf{W}}^{t+1}, \tilde{\mathbf{V}}^{t+1}$

velocity is initialized as

$$\tilde{\mathbf{V}}^2 = \delta \tilde{\mathbf{W}}^1,$$

and the corresponding weight update is given by

$$\Delta \tilde{\mathbf{W}}^1 = (\gamma_{r \times c} \odot_o \delta \tilde{\mathbf{W}}^1) +_\bullet (\gamma \mu_{r \times c} \odot_o \delta \tilde{\mathbf{W}}^1).$$

For subsequent iterations $t \in \{2, \dots, T\}$, the encrypted velocity is updated as

$$\tilde{\mathbf{V}}^{t+1} = (\mu_{r \times c} \odot_o \tilde{\mathbf{V}}^t) +_\bullet \delta \tilde{\mathbf{W}}^t.$$

Consequently, the encrypted weight update becomes

$$\Delta \tilde{\mathbf{W}}^t = (\gamma_{r \times c} \odot_o \delta \tilde{\mathbf{W}}^t) +_\bullet (\gamma \mu_{r \times c} \odot_o \delta \tilde{\mathbf{W}}^t) +_\bullet (\gamma \mu^2_{r \times c} \odot_o \tilde{\mathbf{V}}^t),$$

where $\{\gamma_{r \times c}, \gamma \mu_{r \times c}, \gamma \mu^2_{r \times c}\} \in \mathbb{R}^{r \times c}$ are plaintext matrices filled with the constants γ , $\gamma \mu$, and $\gamma \mu^2$, respectively, to match the dimensions of the encrypted weight gradient and velocity. Finally, the encrypted weights are updated as

$$\tilde{\mathbf{W}}^{t+1} = \tilde{\mathbf{W}}^t -_\bullet \Delta \tilde{\mathbf{W}}^t,$$

where $-_\bullet$ denotes homomorphic element-wise subtraction, as defined in Equation 2.

At the end of ReBoot encrypted weight update procedure, two iterations of *approximate bootstrapping* are performed to refresh the ciphertext modulus q of weights and velocities associated with both the *RE-FC* layer h and the *CE-FC* local classifier h^* . This step enables further encrypted operations beyond the original scheme level l , as explained in Section 3.

4.4 ReBoot Multiplicative Depth

To correctly set CKKS encryption parameters, as explained in Section 3, it is essential to determine the maximum number of consecutive homomorphic multiplications required per training iteration, referred to as the multiplicative depth. Specifically, the multiplicative depth, denoted by τ , depends on the network architecture and

Table 1: Maximum multiplicative depth τ for standard BP and ReBoot. H is the number of hidden layers in the MLP.

Algorithm	Forward pass τ^{fw}	Backward pass τ^{bw}
BP	$\lfloor 2.5 \cdot H \rfloor + \lfloor 1.5 + (H \bmod 2) \rfloor$	$\lfloor 2.5 \cdot H \rfloor + 2$
ReBoot	$\lfloor 2.5 \cdot H \rfloor + \lfloor 1.5 + (H \bmod 2) \rfloor$	$\lfloor 4.5 + 1 - (H \bmod 2) \rfloor$

the learning algorithm, and can be expressed as a function of the number of hidden layers H in the considered MLP. Minimizing the multiplicative depth is critical, as it directly impacts computational and memory efficiency. In this context, to highlight ReBoot’s efficiency, we compare the multiplicative depth required to train an encrypted MLP using our proposed framework with that required by the standard Back-Propagation (BP) algorithm.

In standard BP, encrypted gradients are propagated backward from the output classifier to the input layer. As a result, the maximum multiplicative depth for BP, denoted by τ_{BP} , is reached when updating the first layer $h = 1$. In contrast, ReBoot partitions the training into independent encrypted local-loss blocks, and gradients are propagated only within individual blocks, as explained in Section 4.2. Thus, the maximum multiplicative depth of ReBoot, denoted by τ_{ReBoot} , is determined by the encrypted computation at the deepest layer $h = H$.

Table 1 details the multiplicative depth for both BP and ReBoot, including both the forward and backward passes. Specifically, for a *RE-Block*, the forward pass depth is $\tau^{\text{fw}} = 2$, comprising one multiplication from the *RE-Matmul* of the *RE-FC* layer and one from the *EncryptedPolyReLU* activation function. The backward pass multiplicative depth is $\tau^{\text{bw}} = 4$, consisting of one multiplication from the *RE-Matmul* of the *CE-FC* local classifier, one from the derivative of the activation function *EncryptedPolyReLU*’, and two from the weight update procedure. In contrast, for a *CE-Block*, the forward pass multiplicative depth is $\tau^{\text{fw}} = 3$, with two multiplications from the *CE-Matmul* of the *CE-FC* layer and one from the *EncryptedPolyReLU* activation function. The backward pass depth is $\tau^{\text{bw}} = 5$, comprising two multiplications from the *CE-Matmul* of the *RE-FC* local classifier, one from the derivative of the activation function *EncryptedPolyReLU*’, and two from the weight update procedure. The cumulative multiplicative depth of ReBoot at a generic layer $h \in \mathcal{H}$ can be expressed as $\tau_{\text{ReBoot}}^{\text{fw}_h} = \lfloor 2.5 \cdot h \rfloor$ for the forward computation, and $\tau_{\text{ReBoot}}^{\text{bw}_h} = \lfloor 4.5 + 1 - (h \bmod 2) \rfloor$ for the backward computation. It is worth noting that the total multiplicative depth at layer h is given by $\tau_{\text{ReBoot}}^h = \tau_{\text{ReBoot}}^{\text{fw}_h} + \tau_{\text{ReBoot}}^{\text{bw}_h}$.

The reduction in maximum multiplicative depth achieved by ReBoot compared to BP can thus be quantified as

$$\Delta\tau = \tau_{\text{BP}} - \tau_{\text{ReBoot}} = \lfloor 2.5 \cdot H \rfloor - 3 - (H \bmod 2).$$

This expression formalizes the depth savings and highlights ReBoot’s scalability advantage. As the number of hidden layers H in the NN increases, the maximum depth gap $\Delta\tau$ widens, significantly reducing ciphertext noise accumulation and frequency of bootstrapping.

Table 2: Datasets considered in the experimental campaign.

Dataset	Type	Input size	Classes	Train size	Test size
MNIST [25]	Image	14×14	10	60,000	10,000
Fashion-MNIST [59]	Image	14×14	10	60,000	10,000
Kuzushiji-MNIST [21]	Image	14×14	10	60,000	10,000
C-MNIST [44]	Image	8×8	10	60,000	10,000
T-MNIST [23]	Image	4×4	3	50	3,147
Letter Recognition [52]	Tabular	16	26	16,000	4,000
Breast Cancer [58]	Tabular	30	2	455	114
Heart Disease [34]	Tabular	13	5	242	61
Penguins [33]	Tabular	6	3	275	69
Iris [28]	Tabular	4	3	120	30

Table 3: MLP architectures and encryption parameters. N denotes the polynomial degree, Δ the scaling factor, and $l = \tau_{\text{ReBoot}} + \tau_{\text{bs}}$ the scheme level, where τ_{ReBoot} and τ_{bs} represent the maximum multiplicative depth of ReBoot and the multiplicative depth of the approximate bootstrapping procedure, respectively. The dimension k_h of ReBoot’s encrypted fully-connected hidden layers is indicated in parentheses.

Name	Architecture	N	$l = \tau_{\text{ReBoot}} + \tau_{\text{bs}}$	Δ
<i>eMLP-1</i>	<i>RE-Block</i> (32)	2^{16}	$24 = 8 + 16$	49
<i>eMLP-2</i>	<i>RE-Block</i> (64) $\rightarrow$ <i>CE-Block</i> (32)	2^{17}	$27 = 11 + 16$	59
<i>eMLP-3</i>	<i>RE-Block</i> (128) $\rightarrow$ <i>CE-Block</i> (64) $\rightarrow$ <i>RE-Block</i> (32)	2^{17}	$29 = 13 + 16$	59

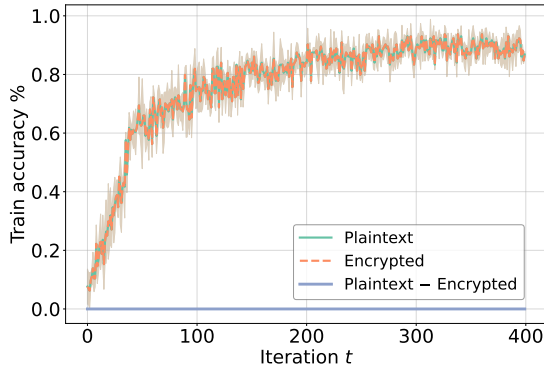
5 Experimental Results

In this section, we evaluate both the effectiveness and efficiency of ReBoot in comparison to existing solutions in the literature. Specifically, Section 5.1 outlines the experimental setup, including the datasets, NN architectures, and CKKS encryption parameters used. Section 5.2 examines the impact of CKKS approximate computation on the precision of the encrypted training process, while Section 5.3 evaluates ReBoot accuracy in comparison to standard MLPs trained using FP32-precision plaintext BP. Section 5.4 presents a comparative analysis of ReBoot accuracy against state-of-the-art (SotA) solutions. Finally, Section 5.5 provides an in-depth assessment of ReBoot computational requirements, benchmarking its training latency against related encrypted DNN frameworks.

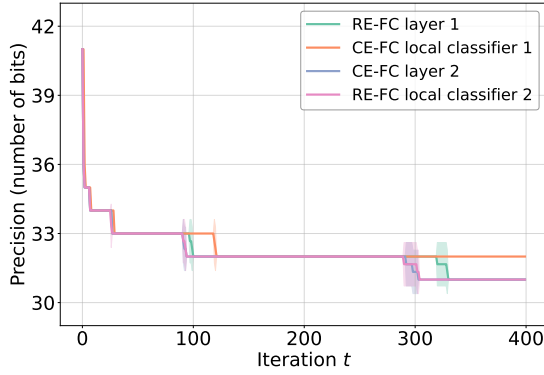
5.1 Experimental Setup

The experimental campaign is based on publicly available benchmarks, comprising 5 image classification datasets and 5 tabular datasets, as listed in Table 2. All experiments were conducted on an Ubuntu 20.04 LTS workstation equipped with two Intel Xeon Gold 5318S CPUs and 384 GB of RAM. The set of hyperparameters used for each experiment are provided in Appendix A. The code implementing ReBoot is made available as a public repository.

To assess ReBoot performance and accuracy, three NN architectures are considered, denoted as *eMLP-1*, *eMLP-2*, and *eMLP-3*. These architectures, along with their corresponding sets of encryption parameters Θ , are detailed in Table 3. The dimensions of the *RE-FC* and *CE-FC* layers are indicated in parentheses. The CKKS parameters Θ are selected to ensure $\lambda = 128$ bits of security, following



(a) ReBoot encrypted and plaintext training accuracy.



(b) Weights precision per layer.

Figure 2: Precision analysis of *eMLP-1* on the MNIST dataset.

the recommendations of the Homomorphic Encryption Standard (HES) [5]. The scheme level l is set to $l = \tau_{\text{ReBoot}} + \tau_{\text{bs}}$, where τ_{ReBoot} represents the maximum multiplicative depth required for training, as explained in Section 4.4, and τ_{bs} accounts for the additional depth needed to perform iterative bootstrapping [8]. The polynomial degree N is chosen as the smallest value that ensures 128-bit security, while the scaling factor Δ is selected to be as large as possible for the chosen N .

5.2 Precision Analysis

In this section, we analyze the impact of the approximate nature of CKKS computations on ReBoot encrypted training. Each homomorphic operation in CKKS introduces noise into ciphertexts, which accumulates over time and progressively degrades their content, as discussed in Section 3. The severity of this effect depends on the scaling factor Δ , which determines the initial precision of the ciphertexts, and the coefficient moduli q , which define how long that precision can be preserved during computation. This is particularly critical because, unlike in FHE schemes, CKKS bootstrapping does not fully refresh noise, making its careful management essential during encrypted training [26].

Figure 2a shows the training accuracy of *eMLP-1* during both ReBoot plaintext and encrypted training on the MNIST dataset, averaged over three independent runs. The exact match between

plaintext and encrypted training accuracy demonstrates that ReBoot encrypted training faithfully replicates the behavior of its FP32 plaintext counterpart, effectively mitigating CKKS approximation errors and managing noise accumulation. Similarly, Figure 2b examines the precision of CKKS ciphertexts representing the weights of *eMLP-1*. Specifically, precision is measured as the number of bits in the encrypted weights that match the corresponding plaintext weights, following the method described in Li et al. [38]. The results show that precision decays logarithmically across layers, with a pronounced drop after the first bootstrapping step. This drop is primarily attributed to noise accumulation and approximation errors introduced during bootstrapping. Nevertheless, after the initial drop, the precision stabilizes in subsequent iterations, enabling ReBoot to sustain effective encrypted training.

These results confirm the functional equivalence between ReBoot encrypted and plaintext training, both in terms of learning dynamics and numerical precision. Given this demonstrated alignment, and considering the significant computational overhead of HE, subsequent ReBoot accuracy experiments are conducted using plaintext training to enable broader and more efficient evaluations across different datasets and configurations.

5.3 ReBoot Test Accuracy

In this section, we evaluate the generalization performance of ReBoot in comparison to standard FP32-precision plaintext BP [50]. To this end, we train both *eMLP* architectures using ReBoot and corresponding plaintext MLPs (i.e., with the same architecture) using standard BP. Specifically, plaintext models employ the NAG [54] optimizer, the *categorical cross-entropy* loss function, and the *ReLU* [2] activation function, all of which are incompatible with CKKS. The complete set of hyperparameters is provided in Appendix A.

Table 4 reports ReBoot’s test accuracy across *eMLP* architectures, compared to their corresponding standard FP32-precision plaintext BP counterparts, averaged over 10 runs. Despite operating entirely under HE constraints, ReBoot consistently achieves accuracy levels comparable to those of HE-incompatible models trained with BP. Moreover, ReBoot’s test accuracy improves with increasing model complexity, indicating that its learning algorithm scales effectively with NN width and depth. These results affirm ReBoot’s ability to deliver accurate and reliable encrypted training without requiring access to plaintext data or intermediate decryption, making it well-suited for privacy-preserving ML *as-a-service* applications.

5.4 Comparison with SotA Solutions

This section evaluates ReBoot’s effectiveness in comparison to related work in the literature. In the context of logistic regression, several studies [9, 24, 36] are limited to small-scale or proof-of-concept scenarios due to the lack of bootstrapping support. In contrast, while Carpov et al. [15] and Han et al. [32] integrate bootstrapping, they do not provide benchmarks on publicly available datasets. Given these limitations, we focus our comparison on the work of Han et al. [32]. Results are reported in Table 5. In particular, the MNIST dataset was reduced to a binary classification task, following the setup described in Han et al. [32]. ReBoot’s ability to train more complex models enables it to outperform encrypted logistic regression, achieving a test accuracy improvement of +3.27% using

Table 4: Test accuracy comparison across 10 independent runs between ReBoot and FP32-precision plaintext BP training.

Dataset	<i>eMLP-1</i>		<i>eMLP-2</i>		<i>eMLP-3</i>	
	ReBoot	FP32 BP	ReBoot	FP32 BP	ReBoot	FP32 BP
MNIST	94.61 $\pm$ 0.27	94.58 $\pm$ 0.46	96.39 $\pm$ 0.16	94.18 $\pm$ 0.32	96.77 $\pm$ 0.13	95.03 $\pm$ 0.36
Fashion-MNIST	85.12 $\pm$ 0.20	85.09 $\pm$ 0.22	86.32 $\pm$ 0.23	82.27 $\pm$ 0.72	86.64 $\pm$ 0.28	86.25 $\pm$ 5.81
Kuzushiji-MNIST	76.60 $\pm$ 0.66	79.41 $\pm$ 0.51	82.31 $\pm$ 0.53	79.57 $\pm$ 0.56	83.83 $\pm$ 0.37	82.07 $\pm$ 0.51
Breast Cancer	99.03 $\pm$ 1.78	98.95 $\pm$ 1.70	98.96 $\pm$ 1.78	98.25 $\pm$ 2.74	99.03 $\pm$ 1.88	98.74 $\pm$ 2.49
Heart Disease	87.06 $\pm$ 5.12	87.62 $\pm$ 5.72	93.44 $\pm$ 2.64	94.87 $\pm$ 3.00	93.25 $\pm$ 2.64	94.13 $\pm$ 2.06
Letter Recognition	73.08 $\pm$ 1.27	75.06 $\pm$ 1.77	85.84 $\pm$ 0.72	85.66 $\pm$ 1.20	90.66 $\pm$ 0.90	89.27 $\pm$ 0.81

the *eMLP-1* architecture. Thanks to ReBoot, encrypted training can go beyond single-layer models, enabling the training of full NNs.

In the context of DNNs, and to ensure a fair comparison with SotA solutions, ReBoot was evaluated using both the original MLP architectures from prior work and *eMLP* models, which match the NN depth H but feature increased layer width. Notably, ReBoot enables the use of wider fully-connected layers while preserving overall computational complexity, thereby allowing the training of more expressive models without additional cost, as analyzed in Section 5.5. Table 5 summarizes the experimental results in terms of test accuracy. ReBoot consistently outperforms prior encrypted DNN methods across all evaluated datasets, with improvements of up to +6.83%. In particular, while the vast majority of existing approaches rely on integer-based training, ReBoot supports real-valued training, enabling a higher-precision learning process.

5.5 Computational and Memory Demands

This section evaluates the computational and memory demands of ReBoot. Specifically, we compare ReBoot’s training latency with that of related encrypted DNN frameworks. In the context of DNNs, Nandakumar et al. [44] and Lou et al. [39] adopt an insecure 80-bit security level, which allows the use of a smaller polynomial degree N , reducing computational cost but falling short of HES recommendations. Similarly, Mihara et al. [41] and Montero et al. [42] replace the bootstrapping procedure, which accounts for up to 86% of the total training iteration time, with a *re-encryption* mechanism. However, this mechanism requires continuous client-server interaction, making it unsuitable for *as-a-service* scenarios. Given these limitations, we focus our comparison on the works of Yoo and Yoon [60] and Colombo et al. [23], which provide a fair and meaningful basis for evaluation.

Table 6 reports the time required to execute a single training iteration t . Specifically, ReBoot was evaluated using both the original MLP architectures from prior work and *eMLP* models. The results highlight ReBoot’s significant performance advantage, achieving speedups ranging from $3.40\times$ to $8.83\times$. This gain is primarily attributed to ReBoot packing, which enables full exploitation of CKKS *SIMD* operations, as explained in Section 4.1. In contrast, the lack of *SIMD* support in TFHE results in sequential execution, with $\mathcal{O}(r \times c)$ complexity for a $r \times c$ weight matrix.

To assess the scalability of ReBoot’s computational demands, we conduct experiments analyzing the impact of three key parameters on training latency: the polynomial degree N , the width of the NN layers k_h , and the overall NN depth H . Figure 3 shows the

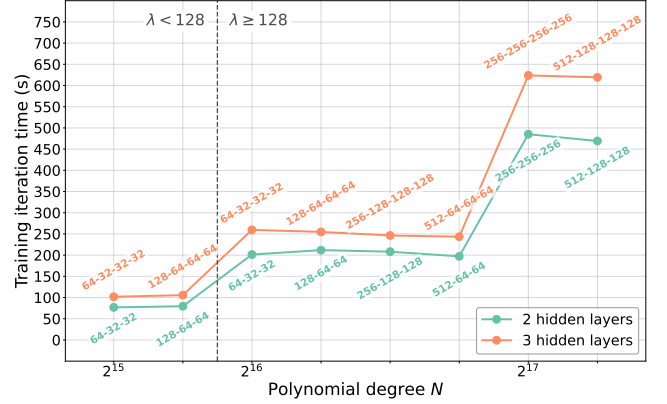


Figure 3: Computational demands of ReBoot encrypted NNs in terms of training iteration time as a function of polynomial degree N , layer width, and NN depth H .

time required to perform a single encrypted training iteration t for ReBoot architectures with $H = 2$ and $H = 3$ hidden layers. Experiments using $N = 2^{15}$ and $N = 2^{16}$ adopt a scaling factor $\Delta = 49$, while those with $N = 2^{17}$ use $\Delta = 59$. It is important to note that configurations with $N = 2^{15}$ do not satisfy the 128-bit security requirement and are included solely for the purpose of evaluating computational performance. Two main observations emerge. First, increasing N significantly impacts latency: doubling N more than doubles the time required per training iteration. Second, for a fixed N , latency remains nearly constant across varying layer widths thanks to ReBoot packing, which leverages CKKS *SIMD* capabilities for parallel computation. As detailed in Section 4.2.1, encrypted matrix multiplications scale logarithmically with the number of neurons. However, when the layer dimension exceeds the number of available ciphertext slots, a larger N is needed, resulting in a substantial increase in iteration time. In contrast, increasing the network depth has only a moderate impact, as it adds relatively few sequential encrypted operations.

Additionally, we analyze the memory usage associated with the *eMLP* architectures presented in Table 3. In general, memory consumption is primarily driven by the number of ciphertexts required to encrypt various components of the training process, including model weights, input data, intermediate activations, gradients, and output predictions. The trained encrypted models occupy 52 MB for

Table 5: Test accuracy comparison between ReBoot and related encrypted training frameworks over 10 runs.

Dataset	Work	HE Scheme	λ	Architecture	Test accuracy (%)
Binary MNIST	Han et al. [32]	CKKS (bootstrapping)	128	Logistic regression	96.40
	ReBoot	CKKS (bootstrapping)	128	<i>eMLP-1</i>	99.67 $\pm$ 0.13
MNIST	Lou et al. [39]	BFV/TFHE	80	MLP[784-128-32-10]	96.60
	ReBoot	CKKS (bootstrapping)	128	MLP[784-128-32-10]	97.67 $\pm$ 0.13
C-MNIST	Nandakumar et al. [44]	BGV	$\ll$ 80	MLP[64-32-16-10]	96.00
	ReBoot	CKKS (bootstrapping)	128	MLP[64-32-16-10]	95.08 $\pm$ 0.13
	ReBoot	CKKS (bootstrapping)	128	<i>eMLP-2</i>	96.71 $\pm$ 0.12
T-MNIST	Colombo et al. [23]	TFHE	128	MLP[16-4-2-3]	91.90
	ReBoot	CKKS (bootstrapping)	128	MLP[16-4-2-3]	91.23 $\pm$ 0.53
	ReBoot	CKKS (bootstrapping)	128	<i>eMLP-1</i>	93.39 $\pm$ 0.64
Fashion-MNIST	Colombo et al. [23]	TFHE	128	MLP[784-200-10]	86.00
	ReBoot	CKKS (bootstrapping)	128	MLP[784-200-10]	89.08 $\pm$ 0.20
Iris	Mihara et al. [41]	CKKS (leveled)	128	MLP[4-10-3]	98.05
	ReBoot	CKKS (bootstrapping)	128	MLP[4-10-3]	98.44 $\pm$ 6.40
	ReBoot	CKKS (bootstrapping)	128	<i>eMLP-1</i>	99.69 $\pm$ 1.88
Penguins	Colombo et al. [23]	TFHE	128	MLP[4-2-3]	92.20
	ReBoot	CKKS (bootstrapping)	128	MLP[4-2-3]	81.18 $\pm$ 2.35
	ReBoot	CKKS (bootstrapping)	128	<i>eMLP-1</i>	99.03 $\pm$ 0.68
Breast Cancer	Montero et al. [42]	TFHE (leveled)	128	MLP[30-29-1]	98.25
	ReBoot	CKKS (bootstrapping)	128	MLP[30-29-1]	98.95 $\pm$ 1.99
	ReBoot	CKKS (bootstrapping)	128	<i>eMLP-1</i>	99.03 $\pm$ 1.78

Table 6: Latency comparison between ReBoot and related encrypted training frameworks.

Work	HE Scheme	λ	Architecture	Training iteration time (s)
Yoo and Yoon [60]	TFHE	128	MLP[1-1-1]	1681.20
ReBoot	CKKS (bootstrapping)	128	MLP[1-1-1]	198.37
ReBoot	CKKS (bootstrapping)	128	<i>eMLP-1</i>	190.32
Colombo et al. [23]	TFHE	128	MLP[4-2-3]	646.72
ReBoot	CKKS (bootstrapping)	128	MLP[4-2-3]	193.77
ReBoot	CKKS (bootstrapping)	128	<i>eMLP-1</i>	190.32
Colombo et al. [23]	TFHE	128	MLP[16-4-2-3]	5034.00
ReBoot	CKKS (bootstrapping)	128	MLP[16-4-2-3]	621.15
ReBoot	CKKS (bootstrapping)	128	<i>eMLP-2</i>	629.65

eMLP-1, 224 MB for *eMLP-2*, and 360 MB for *eMLP-3*. During training, these models reach peak memory usage of 76.07 GB, 181.45 GB, and 213.63 GB, respectively. The substantial increase in memory footprint between *eMLP-1* and *eMLP-2* is primarily due to the use of a larger polynomial degree N . Overall, memory usage scales consistently with both the NN depth and the batch size. As expected, the size of the encrypted model also grows proportionally with the number of layers: each additional layer introduces new ciphertexts not only for its weights but also for tracking momentum during optimization.

In conclusion, these experiments demonstrate that ReBoot advances the SotA in secure and non-interactive encrypted training, with computational cost largely independent of layer width (within ciphertext slot limits) and scaling moderately with NN depth. This

validates ReBoot’s scalability and efficiency for training deep and wide encrypted DNNs.

6 Conclusions

This paper introduced ReBoot, the first framework to enable fully encrypted and non-interactive training of MLPs using CKKS with bootstrapping. ReBoot proposes an advanced packing technique that significantly reduces both computational and memory overhead by leveraging CKKS *SIMD* capabilities. Moreover, it introduces a dedicated HE-compliant encrypted architecture that minimizes the multiplicative depth required during training. Finally, ReBoot advances the SotA by providing a fully encrypted and non-interactive learning algorithm that enables the training of MLPs on encrypted data. Despite operating entirely under HE constraints,

ReBoot achieves accuracy levels comparable to those of plaintext-trained models, demonstrating that encrypted training can be performed without compromising effectiveness.

Future work will focus on extending ReBoot to support the training of more complex DNN architectures, such as convolutional and recurrent NNs, on real-world use cases [22, 56]. Additionally, distributed training will be explored to further reduce training latency by parallelizing the training of each encrypted local-loss block across separate machines, enabling scalable cloud-based infrastructures for privacy-preserving machine learning *as-a-service* applications.

Acknowledgments

The authors would like to thank Aurora A.F. Colombo for valuable suggestions and support throughout the writing process of the ReBoot paper. This paper is supported by Dhiria s.r.l. and by PNRR-PE-AI FAIR project funded by the NextGeneration EU program.

References

- [1] Abbas Acar, Hidayet Aksu, A. Selcuk Uluagac, and Mauro Conti. 2018. A Survey on Homomorphic Encryption Schemes: Theory and Implementation. *ACM Comput. Surv.* 51, 4, Article 79 (jul 2018), 35 pages. <https://doi.org/10.1145/3214303>
- [2] Abien Fred Agarap. 2019. Deep Learning using Rectified Linear Units (ReLU). arXiv:1803.08375 [cs.NE] <https://arxiv.org/abs/1803.08375>
- [3] Ahmad Al Badawi, Louie Hoang, Chan Fook Mun, Kim Laine, and Khin Mi Mi Aung. 2020. Privft: Private and fast text classification with homomorphic encryption. *IEEE Access* 8 (2020), 226544–226556.
- [4] Ahmad Al Badawi and Yuriy Polyakov. 2023. Demystifying bootstrapping in fully homomorphic encryption. *Cryptology ePrint Archive* (2023).
- [5] Martin Albrecht, Melissa Chase, Hao Chen, Jintai Ding, Shafi Goldwasser, Sergey Gorbunov, Shai Halevi, Jeffrey Hoffstein, Kim Laine, Kristin Lauter, Satya Lokam, Daniele Micciancio, Dustin Moody, Travis Morrison, Amit Sahai, and Vinod Vaikuntanathan. 2019. Homomorphic Encryption Standard. *Cryptology ePrint Archive*, Paper 2019/939. <https://eprint.iacr.org/2019/939>
- [6] Ramy E. Ali, Jinhyun So, and A. Salman Avestimehr. 2024. On Polynomial Approximations for Privacy-Preserving and Verifiable ReLU Networks. arXiv:2011.05530 [cs.LG] <https://arxiv.org/abs/2011.05530>
- [7] Ahmad Al Badawi, Andreea Alexandru, Jack Bates, Flavio Bergamaschi, David Bruce Cousins, Saroja Erabelli, Nicholas Genise, Shai Halevi, Hamish Hunt, Andrey Kim, Yongwoo Lee, Zeyu Liu, Daniele Micciancio, Carlo Pascoe, Yuriy Polyakov, Ian Quah, Saraswathy R.V., Kurt Rohloff, Jonathan Saylor, Dmitriy Sponitsky, Matthew Triplett, Vinod Vaikuntanathan, and Vincent Zucca. 2022. OpenFHE: Open-Source Fully Homomorphic Encryption Library. *Cryptology ePrint Archive*, Paper 2022/915. <https://eprint.iacr.org/2022/915>
- [8] Youngjin Bae, Jung Hee Cheon, Wonhee Cho, Jaehyung Kim, and Taekyung Kim. 2022. META-BTS: Bootstrapping Precision Beyond the Limit. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security* (Los Angeles, CA, USA) (CCS '22). Association for Computing Machinery, New York, NY, USA, 223–234. <https://doi.org/10.1145/3548606.3560696>
- [9] Flavio Bergamaschi, Shai Halevi, Tzipora T Halevi, and Hamish Hunt. 2019. Homomorphic training of 30,000 logistic regression models. In *Applied Cryptography and Network Security: 17th International Conference, ACNS 2019, Bogota, Colombia, June 5–7, 2019, Proceedings 17*. Springer, 592–611.
- [10] Christopher M. Bishop. 1995. Neural networks for pattern recognition. <https://api.semanticscholar.org/CorpusID:60563397>
- [11] Fabian Boemer, Anamaria Costache, Rosario Cammarota, and Casimir Wierzynski. 2019. nGraph-HE2: A High-Throughput Framework for Neural Network Inference on Encrypted Data. In *Proceedings of the 7th ACM Workshop on Encrypted Computing & Applied Homomorphic Cryptography* (London, United Kingdom) (WAHC '19). Association for Computing Machinery, New York, NY, USA, 45–56. <https://doi.org/10.1145/3338469.3358944>
- [12] Jean-Philippe Bossuat, Christian Mouchet, Juan Troncoso-Pastoriza, and Jean-Pierre Hubaux. 2021. Efficient Bootstrapping for Approximate Homomorphic Encryption with Non-sparse Keys. In *Advances in Cryptology – EUROCRYPT 2021*, Anne Canteaut and François-Xavier Standaert (Eds.). Springer International Publishing, Cham, 587–617.
- [13] Zvika Brakerski, Craig Gentry, and Vinod Vaikuntanathan. 2012. (Leveled) fully homomorphic encryption without bootstrapping. In *Proceedings of the 3rd Innovations in Theoretical Computer Science Conference* (Cambridge, Massachusetts) (ITCS '12). Association for Computing Machinery, New York, NY, USA, 309–325. <https://doi.org/10.1145/2090236.2090262>
- [14] Zvika Brakerski, Adeline Langlois, Chris Peikert, Oded Regev, and Damien Stehlé. 2013. Classical hardness of learning with errors. In *Proceedings of the forty-fifth annual ACM symposium on Theory of computing*, 575–584.
- [15] Sergiu Carpop, Nicolas Gama, Mariya Georgieva, and Juan Ramon Troncoso-Pastoriza. 2019. Privacy-preserving semi-parallel logistic regression training with Fully Homomorphic Encryption. *Cryptology ePrint Archive*, Paper 2019/101. <https://eprint.iacr.org/2019/101>
- [16] Giuliano Casale and Manuel Roveri. 2023. Scheduling inputs in early exit neural networks. *IEEE Trans. Comput. Ding*, 2 (2023), 451–465.
- [17] Melissa Chase, Hao Chen, Jintai Ding, Shafi Goldwasser, Sergey Gorbunov, Jeffrey Hoffstein, Kristin Lauter, Satya Lokam, Dustin Moody, Travis Morrison, et al. 2017. Security of homomorphic encryption. *HomomorphicEncryption.org, Redmond WA, Tech. Rep* (2017).
- [18] Jung Hee Cheon, Kyoohyung Han, Andrey Kim, Miran Kim, and Yongsoo Song. 2018. Bootstrapping for approximate homomorphic encryption. In *Advances in Cryptology – EUROCRYPT 2018: 37th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Tel Aviv, Israel, April 29–May 3, 2018 Proceedings, Part I 37*. Springer, 360–384.
- [19] Jung Hee Cheon, Andrey Kim, Miran Kim, and Yongsoo Song. 2017. Homomorphic Encryption for Arithmetic of Approximate Numbers. In *International Conference on the Theory and Application of Cryptology and Information Security*. <https://api.semanticscholar.org/CorpusID:3164123>
- [20] Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène. 2018. TFHE: Fast Fully Homomorphic Encryption over the Torus. *Cryptology ePrint Archive*, Paper 2018/421. <https://eprint.iacr.org/2018/421>
- [21] Tarin Clanuwat, Mikel Bober-Irizar, Asanobu Kitamoto, Alex Lamb, Kazuaki Yamamoto, and David Ha. 2018. Deep Learning for Classical Japanese Literature. arXiv:cs.CV/1812.01718 [cs.CV]
- [22] Aurora AF Colombo, Luca Colombo, Alessandro Falcetta, and Manuel Roveri. 2024. Enhancing Privacy-Preserving Cancer Classification with Convolutional Neural Networks. In *Biocomputing 2025: Proceedings of the Pacific Symposium*. World Scientific, 565–579.
- [23] Luca Colombo, Alessandro Falcetta, and Manuel Roveri. 2024. Training Encrypted Neural Networks on Encrypted Data with Fully Homomorphic Encryption. In *Proceedings of the 12th Workshop on Encrypted Computing & Applied Homomorphic Cryptography* (Salt Lake City, UT, USA) (WAHC '24). Association for Computing Machinery, New York, NY, USA, 64–75. <https://doi.org/10.1145/3689945.3694802>
- [24] Eric Crockett. 2020. A low-depth homomorphic circuit for logistic regression model training. *Cryptology ePrint Archive* (2020).
- [25] Li Deng. 2012. The mnist database of handwritten digit images for machine learning research. *IEEE Signal Processing Magazine* 29, 6 (2012), 141–142.
- [26] Nir Drucker, Guy Moshkovich, Tomer Pelleg, and Hayim Shaul. 2022. BLEACH: Cleaning Errors in Discrete Computations over CKKS. *Cryptology ePrint Archive*, Paper 2022/1298. <https://eprint.iacr.org/2022/1298>
- [27] Junfeng Fan and Frederik Vercauteren. 2012. Somewhat Practical Fully Homomorphic Encryption. *Cryptology ePrint Archive*, Paper 2012/144. <https://eprint.iacr.org/2012/144>
- [28] R. A. FISHER. 1936. THE USE OF MULTIPLE MEASUREMENTS IN TAXONOMIC PROBLEMS. *Annals of Eugenics* 7, 2 (1936), 179–188. <https://doi.org/10.1111/j.1469-1809.1936.tb02137.x> arXiv:https://onlinelibrary.wiley.com/doi/pdf/10.1111/j.1469-1809.1936.tb02137.x
- [29] M.J. Flynn. 1966. Very high-speed computing systems. *Proc. IEEE* 54, 12 (1966), 1901–1909. <https://doi.org/10.1109/PROC.1966.5273>
- [30] Craig Gentry. 2009. Fully homomorphic encryption using ideal lattices. In *Proceedings of the Forty-First Annual ACM Symposium on Theory of Computing* (Bethesda, MD, USA) (STOC '09). Association for Computing Machinery, New York, NY, USA, 169–178. <https://doi.org/10.1145/1536414.1536440>
- [31] Shafi Goldwasser and Silvio Micali. 2019. Probabilistic encryption & how to play mental poker keeping secret all partial information. In *Providing sound foundations for cryptography: on the work of Shafi Goldwasser and Silvio Micali*. 173–201.
- [32] Kyoohyung Han, Seungwan Hong, Jung Hee Cheon, and Daejun Park. 2018. Efficient logistic regression on large encrypted data. *Cryptology ePrint Archive* (2018).
- [33] Allison M Horst, Alison Presmanes Hill, and Kristen B Gorman. 2020. *allison-horst/palmerpenguins: v0.1.0*. <https://doi.org/10.5281/zenodo.3960218>
- [34] Andras Janosi, William Steinbrunn, Matthias Pfisterer, and Robert Detrano. 1989. Heart Disease. UCI Machine Learning Repository. DOI: <https://doi.org/10.24432/C52P4X>
- [35] Chao Jin, Mohamed Ragab, and Khin Mi Mi Aung. 2020. Secure transfer learning for machine fault diagnosis under different operating conditions. In *International Conference on Provable Security*. Springer, 278–297.

- [36] Andrey Kim, Yongsoo Song, Miran Kim, Keewoo Lee, and Jung Hee Cheon. 2018. Logistic regression model training based on the approximate homomorphic encryption. *BMC medical genomics* 11 (2018), 23–31.
- [37] Seewoo Lee, Garam Lee, Jung Woo Kim, Junbum Shin, and Mun-Kyu Lee. 2023. HETAL: efficient privacy-preserving transfer learning with homomorphic encryption. In *International Conference on Machine Learning*. PMLR, 19010–19035.
- [38] Baiyu Li, Daniele Micciancio, Mark Schultz-Wu, and Jessica Sorrell. 2022. Securing Approximate Homomorphic Encryption Using Differential Privacy. In *Advances in Cryptology – CRYPTO 2022*, Yevgeniy Dodis and Thomas Shrimpton (Eds.). Springer Nature Switzerland, Cham, 560–589.
- [39] Qian Lou, Bo Feng, Geoffrey Charles Fox, and Lei Jiang. 2020. Glyph: Fast and Accurately Training Deep Neural Networks on Encrypted Data. In *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin (Eds.), Vol. 33. Curran Associates, Inc., 9193–9202. https://proceedings.neurips.cc/paper_files/paper/2020/file/685ac8cad1be5ac98da9556bc1c8d9e-Paper.pdf
- [40] Vadim Lyubashevsky, Chris Peikert, and Oded Regev. 2010. On Ideal Lattices and Learning with Errors over Rings. *J. ACM* 60 (2010), 43:1–43:35. <https://api.semanticscholar.org/CorpusID:1606347>
- [41] Kentaro Mihara, Ryohei Yamaguchi, Miguel Mitsuishi, and Yusuke Maruyama. 2020. Neural Network Training With Homomorphic Encryption. arXiv:2012.13552 [cs.CR] <https://arxiv.org/abs/2012.13552>
- [42] Luis Montero, Jordan Frery, Celia Kherfallah, Roman Bredehoft, and Andrei Stoian. 2024. Neural Network Training on Encrypted Data with TFHE. *arXiv preprint arXiv:2401.16136* (2024).
- [43] Hesham Mostafa, Vishwajith Ramesh, and Gert Cauwenberghs. 2018. Deep supervised learning using local errors. *Frontiers in neuroscience* 12 (2018), 608.
- [44] Karthik Nandakumar, Nalini Ratha, Sharath Pankanti, and Shai Halevi. 2019. Towards Deep Neural Network Training on Encrypted Data. In *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*. 40–48. <https://doi.org/10.1109/CVPRW.2019.00011>
- [45] Arild Nøklund and Lars Hiller Eidnes. 2019. Training Neural Networks with Local Error Signals. arXiv:1901.06656 [stat.ML]
- [46] Monique Ogburn, Claude Turner, and Pushkar Dahal. 2013. Homomorphic encryption. *Procedia Computer Science* 20 (2013), 502–509.
- [47] Prajwal Panzade, Daniel Takabi, and Zhipeng Cai. 2024. I can’t see it but I can Fine-tune it: On Encrypted Fine-tuning of Transformers using Fully Homomorphic Encryption. arXiv:2402.09059 [cs.LG] <https://arxiv.org/abs/2402.09059>
- [48] Adeetya Patel, Michael Eickenberg, and Eugene Belilovsky. 2023. Local learning with neuron groups. *arXiv preprint arXiv:2301.07635* (2023).
- [49] Alberto Pirillo, Luca Colombo, and Manuel Roveri. 2024. NITRO-D: Native Integer-only Training of Deep Convolutional Neural Networks. arXiv:2407.11698 [cs.LG] <https://arxiv.org/abs/2407.11698>
- [50] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. 1986. Learning representations by back-propagating errors. *Nature* 323 (1986), 533–536. <https://api.semanticscholar.org/CorpusID:205001834>
- [51] Simone Scardapane, Michele Scarpiniti, Enzo Baccarelli, and Aurelio Uncini. 2020. Why should we add early exits to neural networks? *Cognitive Computation* 12, 5 (2020), 954–966.
- [52] David Slate. 1991. Letter Recognition. UCI Machine Learning Repository. DOI: <https://doi.org/10.24432/C5ZP40>.
- [53] N. P. Smart and F. Vercauteren. 2014. Fully homomorphic SIMD operations. *Des. Codes Cryptography* 71, 1 (apr 2014), 57–81. <https://doi.org/10.1007/s10623-012-9720-4>
- [54] Ilya Sutskever, James Martens, George Dahl, and Geoffrey Hinton. 2013. On the importance of initialization and momentum in deep learning. In *Proceedings of the 30th International Conference on International Conference on Machine Learning - Volume 28* (Atlanta, GA, USA) (ICML’13). JMLR.org, III–1139–III–1147.
- [55] Surat Teerapittayanon, Bradley McDanel, and Hsiang-Tsung Kung. 2016. Branchynet: Fast inference via early exiting from deep neural networks. In *2016 23rd international conference on pattern recognition (ICPR)*. IEEE, 2464–2469.
- [56] Vele Tosevski and Glenn Gulak. 2025. Large-Scale Recurrent Neural Networks with Fully Homomorphic Encryption for Privacy-Enhanced Speaker Identification. In *ICASSP 2025-2025 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 1–5.
- [57] Roman Walch, Samuel Sousa, Lukas Helminger, Stefanie Lindstaedt, Christian Rechberger, and Andreas Trügler. 2022. Cryptotl: Private, efficient and secure transfer learning. *arXiv preprint arXiv:2205.11935* (2022).
- [58] William Wolberg, Olvi Mangasarian, Nick Street, and W. Street. 1993. Breast Cancer Wisconsin (Diagnostic). UCI Machine Learning Repository. DOI: <https://doi.org/10.24432/C5DW2B>.
- [59] Han Xiao, Kashif Rasul, and Roland Vollgraf. 2017. Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms. arXiv:1708.07747 [cs.LG]
- [60] Joon Soo Yoo and Ji Won Yoon. 2021. t-BMPNet: Trainable Bitwise Multilayer Perceptron Neural Network over Fully Homomorphic Encryption Scheme. *Security and Communication Networks* 2021, 1 (2021), 7621260.
- [61] Linlin Zhang, Hideo Saito, Liang Yang, and Jiajie Wu. 2022. Privacy-preserving federated transfer learning for driver drowsiness detection. *IEEE Access* 10 (2022), 80565–80574.

A Experiments Hyperparameters

This section reports the hyperparameter configurations used in the experiments of Section 5. All ReBoot *eMLPs* employ the components introduced in Section 4, namely the ReBoot encrypted architecture, the ReBoot encrypted learning algorithm with a fixed momentum $\mu = 0.9$, the ReBoot encrypted loss function, and the *Encrypted-PolyReLU*. In contrast, MLPs trained using FP32-precision plaintext BP utilize the NAG [54] optimizer, the *categorical cross-entropy* loss function, and the *ReLU* [2] activation function. Specifically, Table 7 and Table 8 summarize the hyperparameters used in Section 5.4 and Section 5.3, respectively.

Table 7: Hyperparameters of ReBoot for experiments of Table 4. γ denotes the learning rate, η the weight decay rate, and b the batch size.

Dataset	Architecture	γ	η	b
Binary MNIST	<i>eMLP-1</i>	0.001	0.0	48
MNIST	MLP[784-128-32-10]	0.0005	0.001	60
C-MNIST	MLP[64-32-16-10]	0.0005	0.001	48
	<i>eMLP-2</i>	0.0005	0.001	60
T-MNIST	MLP[16-4-2-3]	0.001	0.001	10
	<i>eMLP-1</i>	0.001	0.0	8
Fashion-MNIST	MLP[784-200-100]	0.00005	0.0	48
Iris	MLP[4-10-3]	0.005	0.0	8
	<i>eMLP-1</i>	0.005	0.0	8
Penguins	MLP[4-2-3]	0.005	0.001	16
	<i>eMLP-1</i>	0.0005	0.0	8
Breast Cancer	MLP[30-29-1]	0.005	0.0	16
	<i>eMLP-1</i>	0.001	0.01	16

Table 8: Hyperparameters used for both ReBoot *eMLPs* and standard MLPs trained with FP32-precision plaintext BP in the experiments of Table 4. γ denotes the learning rate, η the weight decay rate, and b the batch size.

Dataset	Algorithm	<i>eMLP-1</i>			<i>eMLP-2</i>			<i>eMLP-3</i>		
		γ	η	b	γ	η	b	γ	η	b
MNIST	ReBoot	0.001	0.001	48	0.001	0.005	48	0.0005	0.001	48
	FP32 BP	0.005	0.001	48	0.001	0.005	48	0.0005	0.001	48
Fashion-MNIST	ReBoot	0.0005	0.0	48	0.0005	0.0	48	0.001	0.005	48
	FP32 BP	0.005	0.001	48	0.0005	0.005	48	0.0005	0.001	48
Kuzushiji-MNIST	ReBoot	0.0005	0.01	48	0.0005	0.01	48	0.0001	0.01	48
	FP32 BP	0.005	0.001	48	0.0005	0.0	48	0.0005	0.0005	48
Breast Cancer	ReBoot	0.005	0.0	16	0.005	0.0	8	0.005	0.0	8
	FP32 BP	0.005	0.0	16	0.001	0.01	16	0.001	0.01	16
Heart Disease	ReBoot	0.0001	0.01	8	0.01	0.0	8	0.005	0.0	8
	FP32 BP	0.01	0.005	8	0.005	0.0	8	0.005	0.0	8
Letter Recognition	ReBoot	0.001	0.001	48	0.005	0.001	48	0.001	0.0	48
	FP32 BP	0.005	0.001	48	0.005	0.001	48	0.001	0.0	48